

Design REST API



Somkiat
Home

Somkiat Puisungnoen

Update Info 1

View Activity Log 10+

...

Timeline

About

Friends 3,138

Photos

More ▾

When did you work at Opendream?

×

...

22 Pending Items

Intro

Software Craftsmanship

Software Practitioner at สยามชำนานุกิจ พ.ศ. 2556

Agile Practitioner and Technical at SPRINT3r

Post

Photo/Video

Live Video

Life Event

What's on your mind?

Public ▾

Post

Somkiat Puisungnoen

15 mins · Bangkok · 🌐 ▾

Java and Bigdata

...



Facebook interface for the page **somkiat.cc**. The top navigation bar includes the Facebook logo, the page name, a search bar, and icons for Home, Messages, Notifications (3), Insights, Publishing Tools, Settings, and Help.

The main content area features a large video player showing a man in a white Superman t-shirt with "SOMKIAT.CC" printed on it, posing against a white background. Below the video are buttons for "Liked", "Following", "Share", and a menu icon. A blue call-to-action button reads "Help people take action on this Page." with a close icon. Another blue button below it says "+ Add a Button".

The left sidebar contains the page name **somkiat.cc**, the handle **@somkiat.cc**, and a menu with options: Home, Posts, Videos, and Photos.



Design REST API



Basic of REST

REST (REpresentational State Transfer)

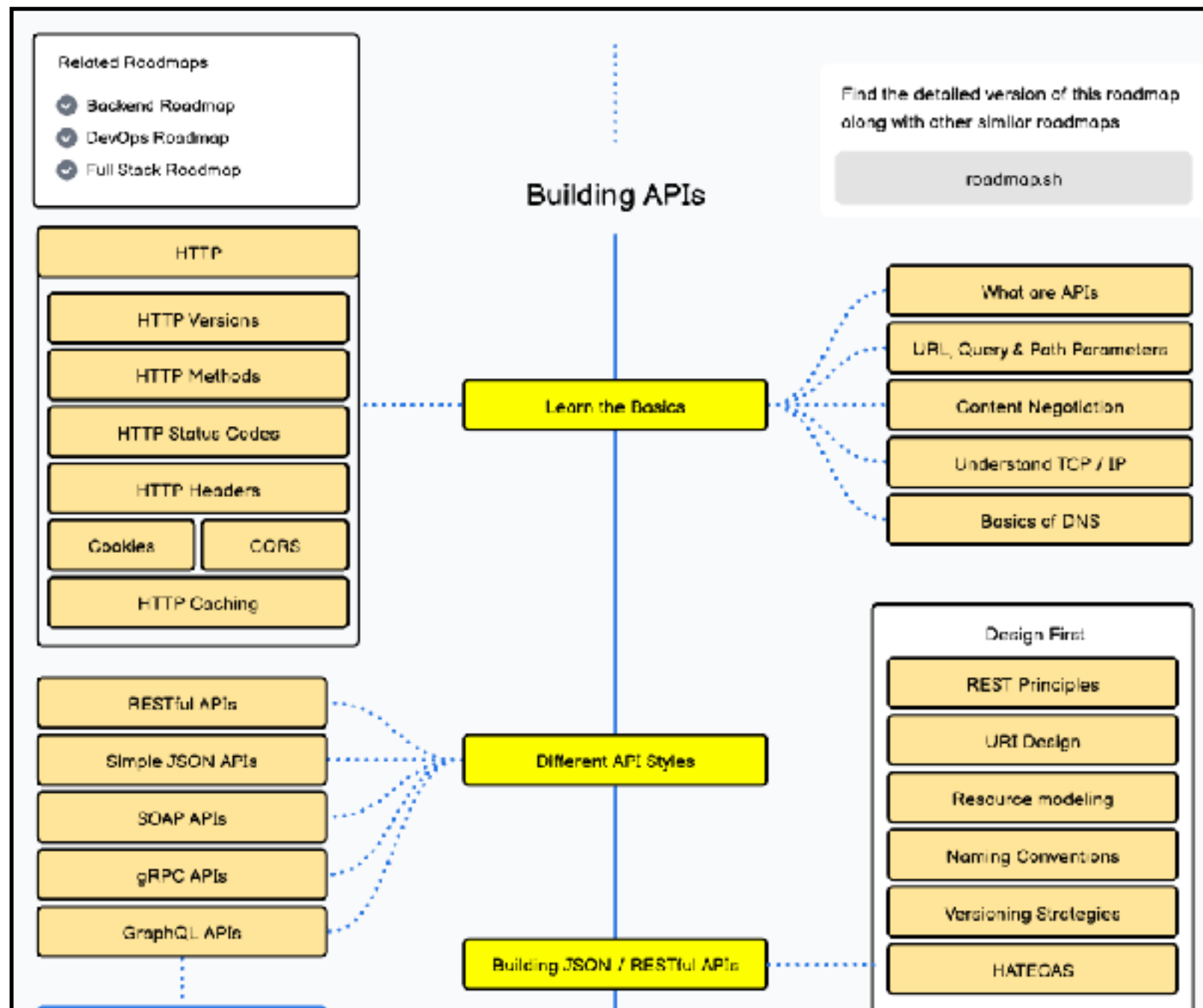
6 REST constraints

REST vs RESTful API

Design principles



REST API Roadmap



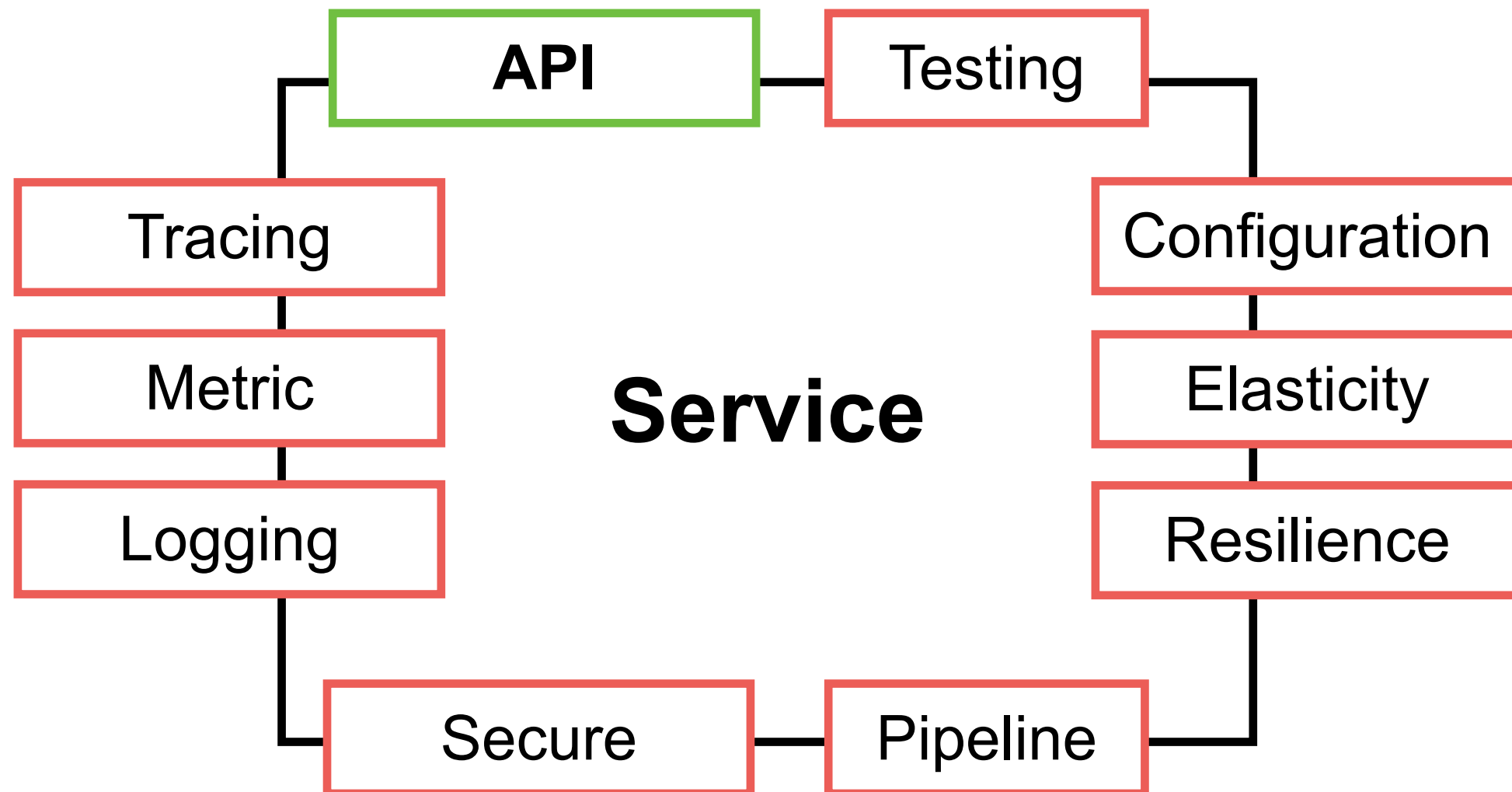
<https://roadmap.sh/api-design>



REST API



Properties of Service



APIs

Application Programming Interface

REST API

gRPC

Messaging



REST

REpresentation State Transfer

The style of **software architecture** behind
RESTful services

Defined in 2000 by Roy Fielding

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm



Goals

Scalability

Generality of interfaces

Independent deployment of components



REST properties

Performance

Scalability

Simplicity

Modification

Visibility

Portability

Reliability



REST constraints

Client-server

Stateless

Cacheable

Layer system

Uniform
interface

Code on
demand

eg. JavaScript



RESTful service

Implementation from REST



REST Request Messages

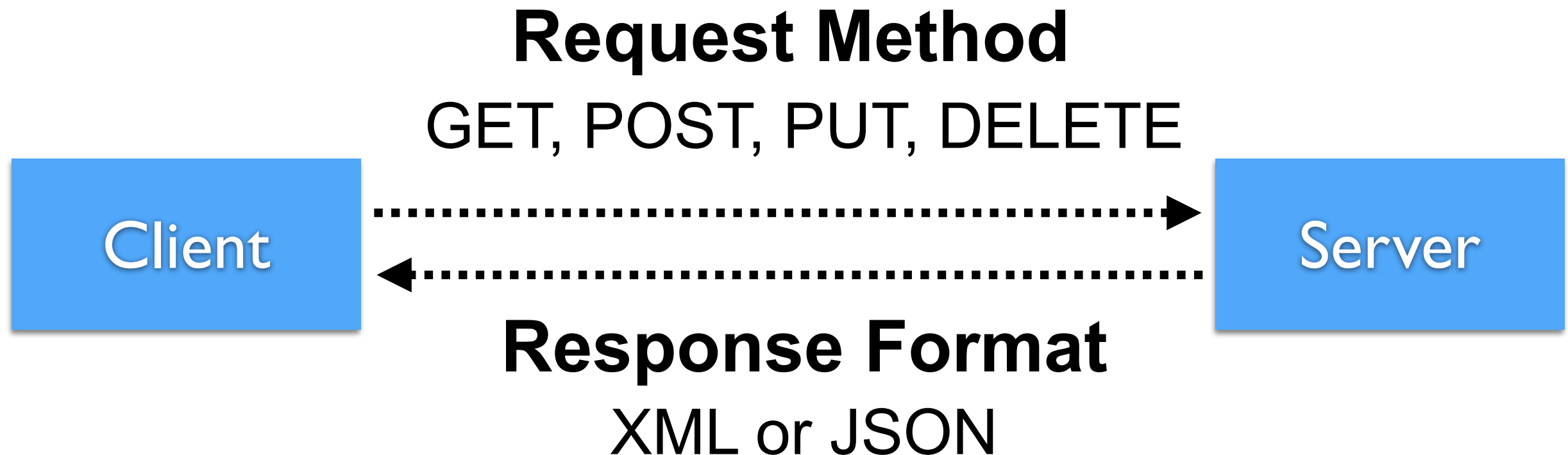
RESTful request is typically in form of
Uniform Resource Identifiers (URI)

Structure of URI depend on specific service

Request can include parameter and data in
body of request as XML, JSON etc.



REST Request & Response



REST Design First !!

URI

Resource
modeling

Naming

Versioning

Filter

Sorting

Problem
detail



HTTP Methods meaning

Method	Meaning
GET	Read data
POST	Create/Insert new data
PUT/PATCH	Update data or insert if a new id
DELETE	Delete data



Response format ?



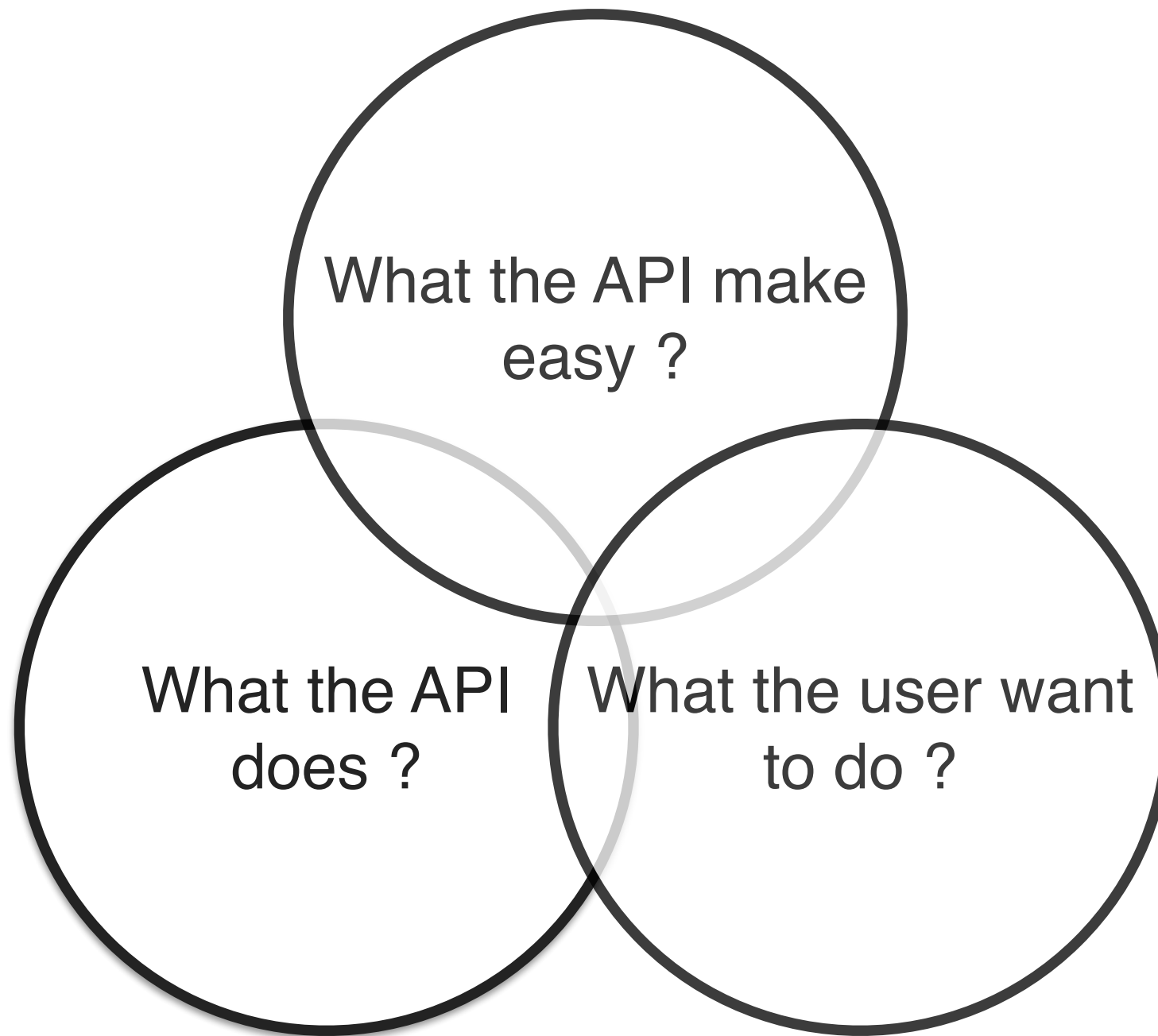
Status Code

Code	Description
1xx	Information
2xx	Successful
3xx	Redirection
4xx	Client error
5xx	Server error

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>



Good APIs ?



Principles of API Design

Consistency

Statelessness

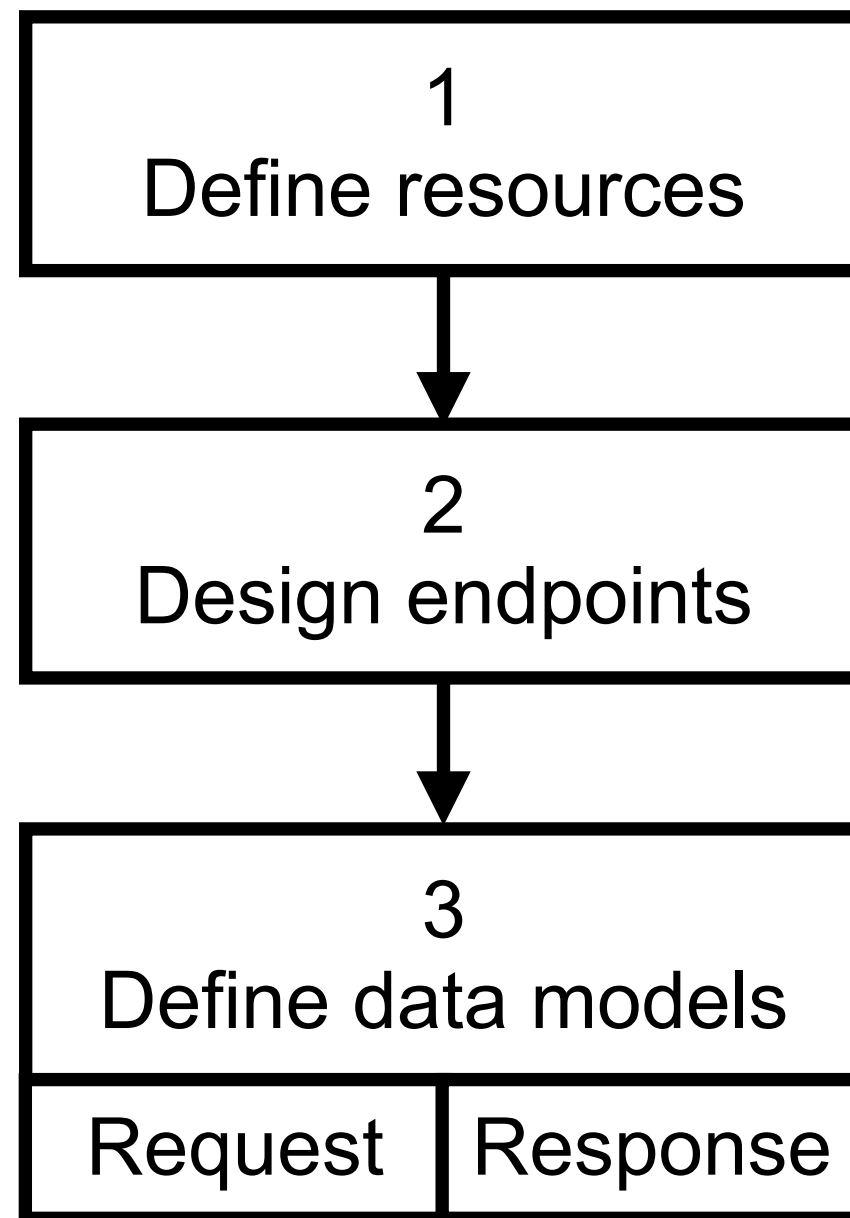
Resource-oriented design

Use standard HTTP methods

Versioning



Steps to Design APIs



Think about test !!



More !!

Authentication
Authorization

Pagination and
filtering

Rate Limiting

Pagination and
filtering

Error handling

Documentation

Testing

Monitoring and
Observability



Implementation

Coding

Testing

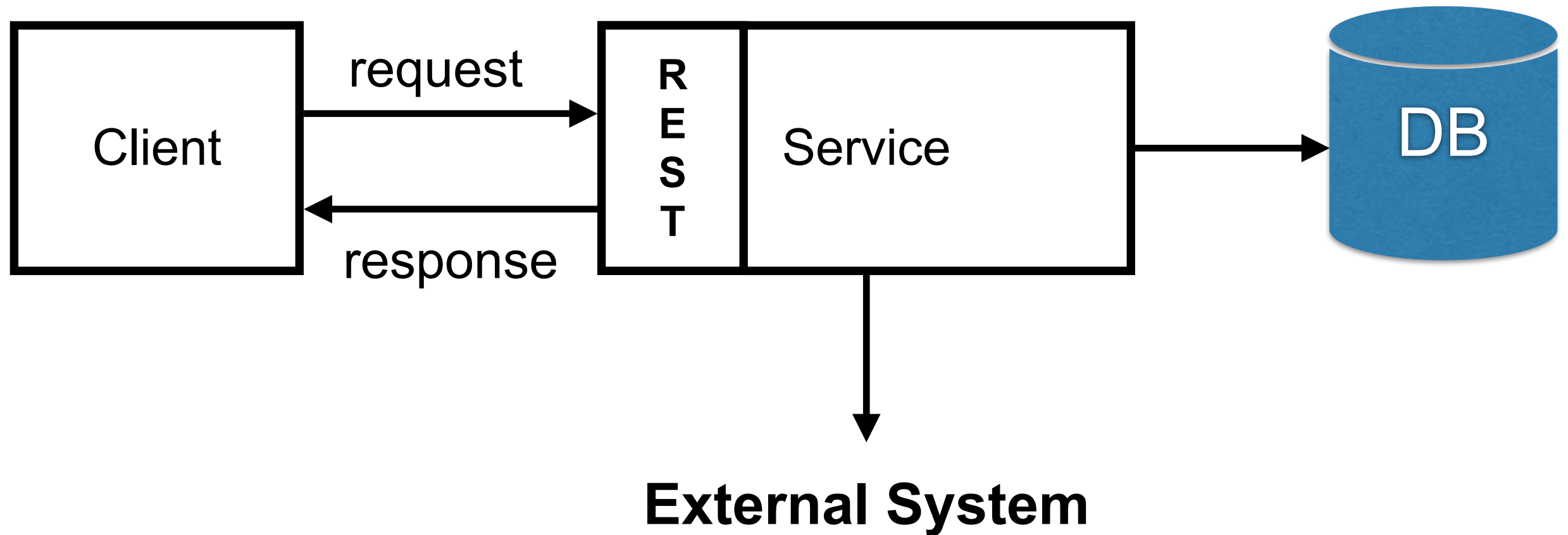
Observability

Performance

Security



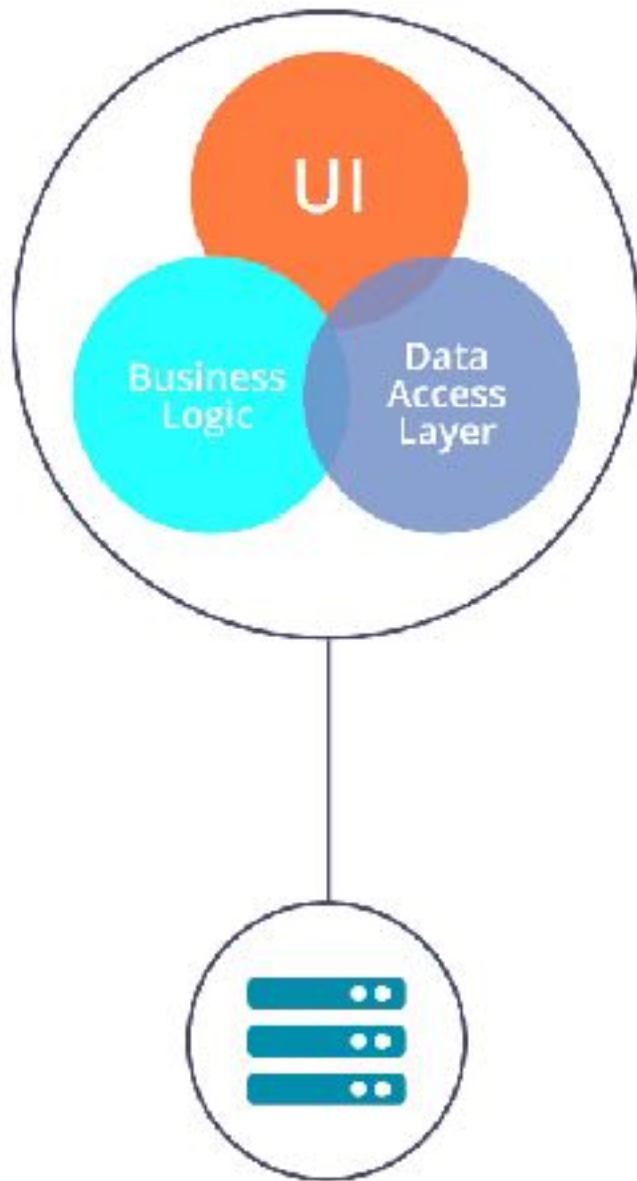
Structure of Service



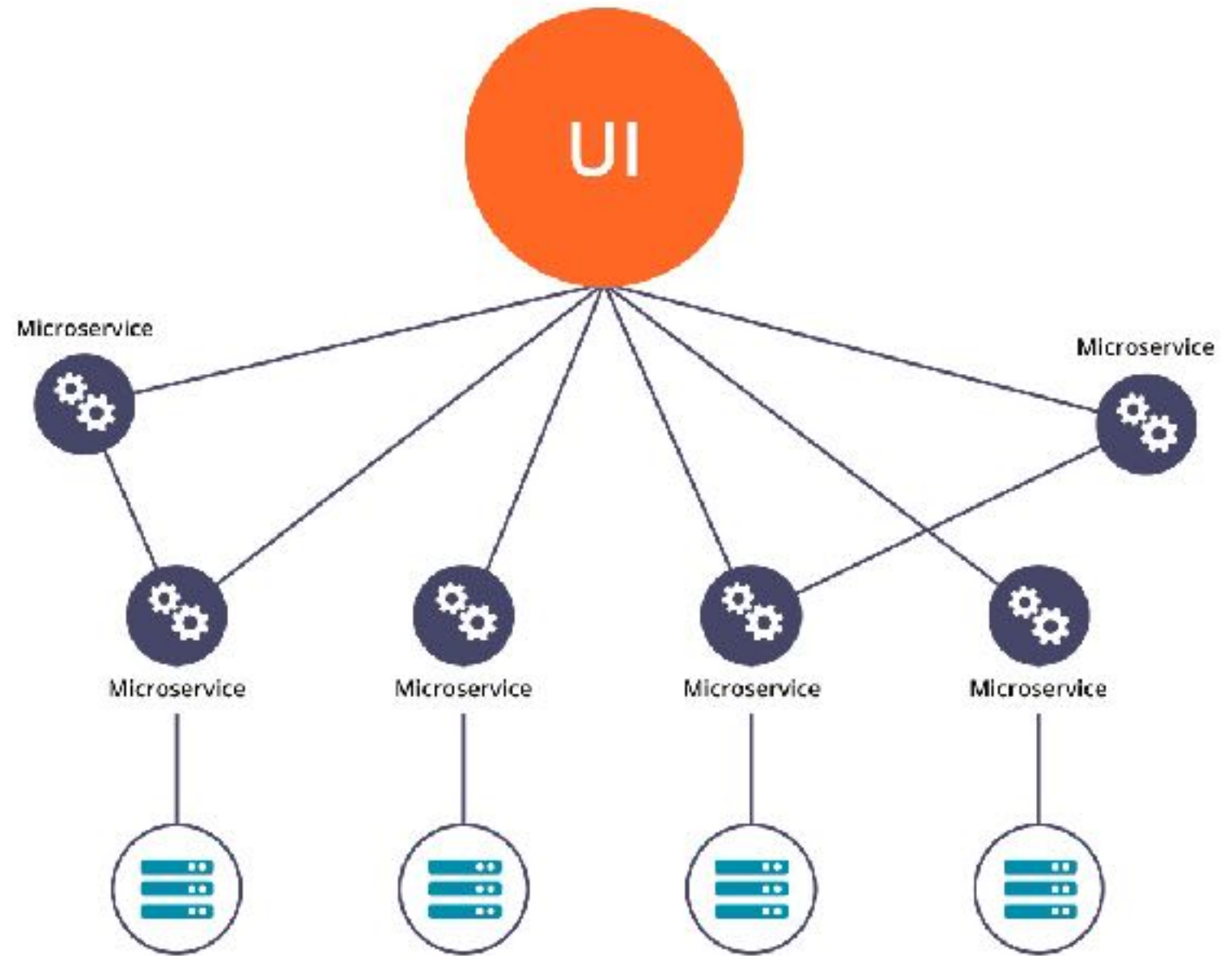
Evolution of Architecture



Monolith to Microservices



Monolithic Architecture

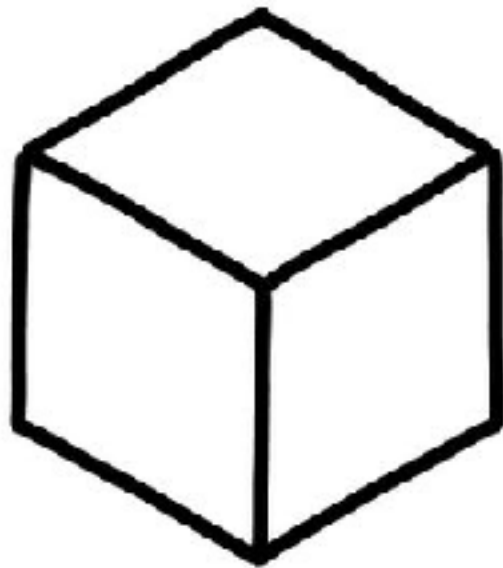


Microservice Architecture



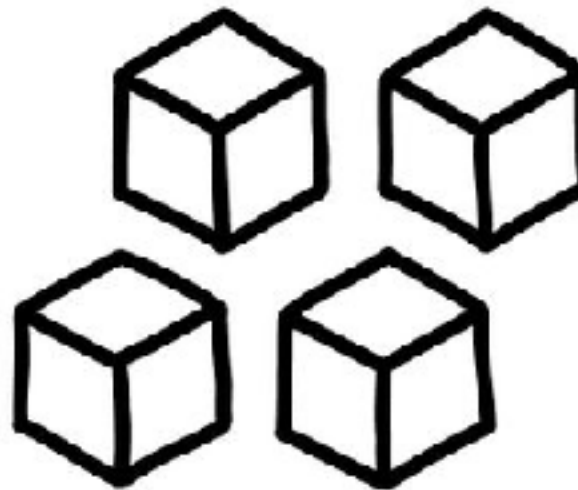
GRANULARITY OF SYSTEMS

Monolithic



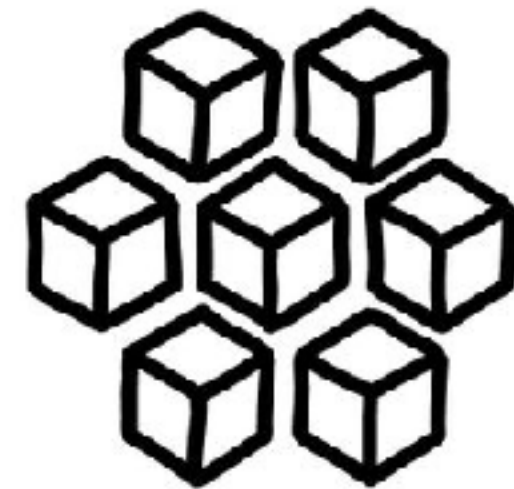
Single unit

SOA



Coarse-grained

Microservices



Fine-grained

<https://architecturenotes.co/p/granularity-of-systems>



Database Design !!

Design for read

Design for write

Pre-join tables or pre-aggregate data

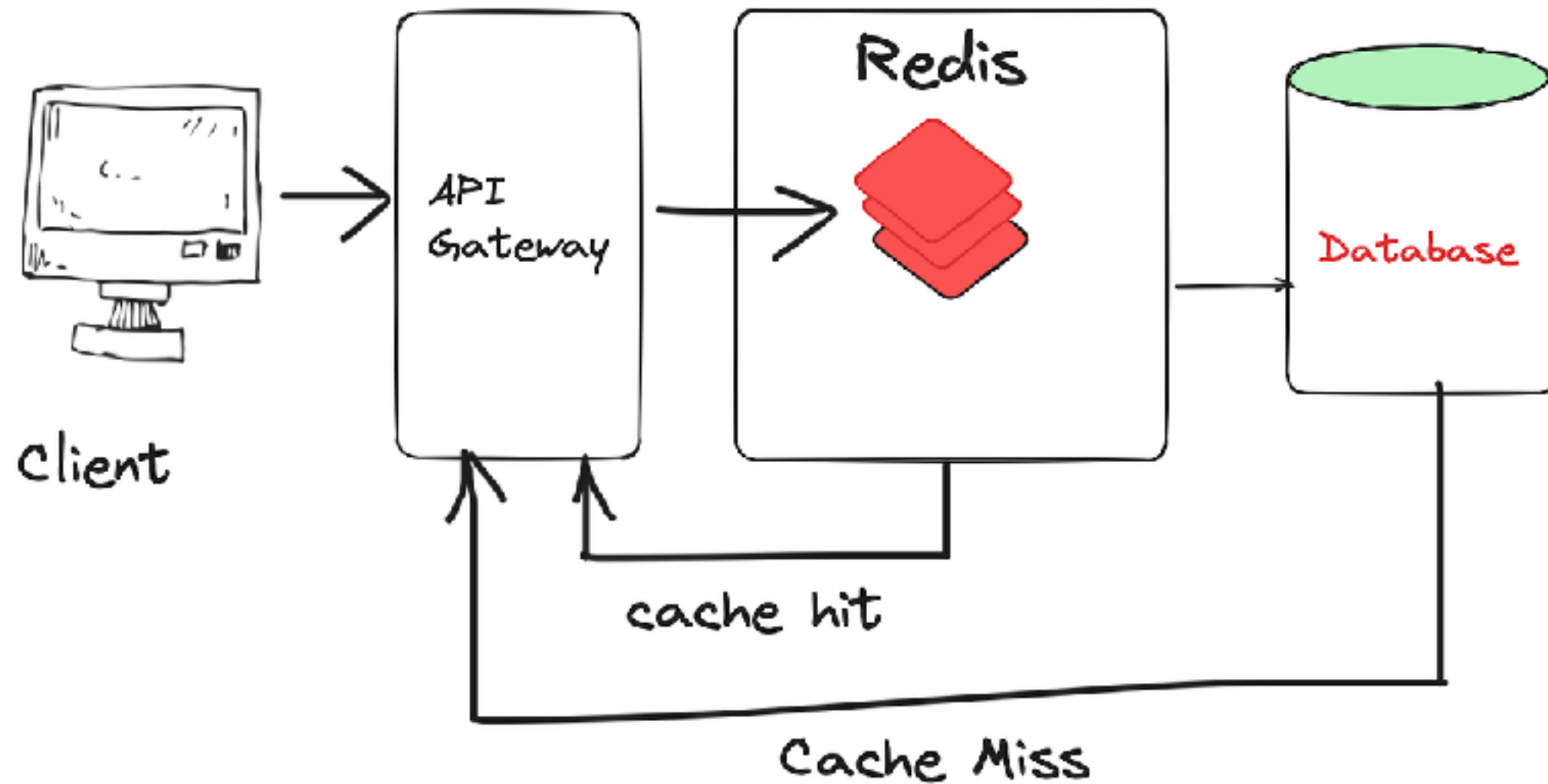
Manage data size

Partitioning and Sharding

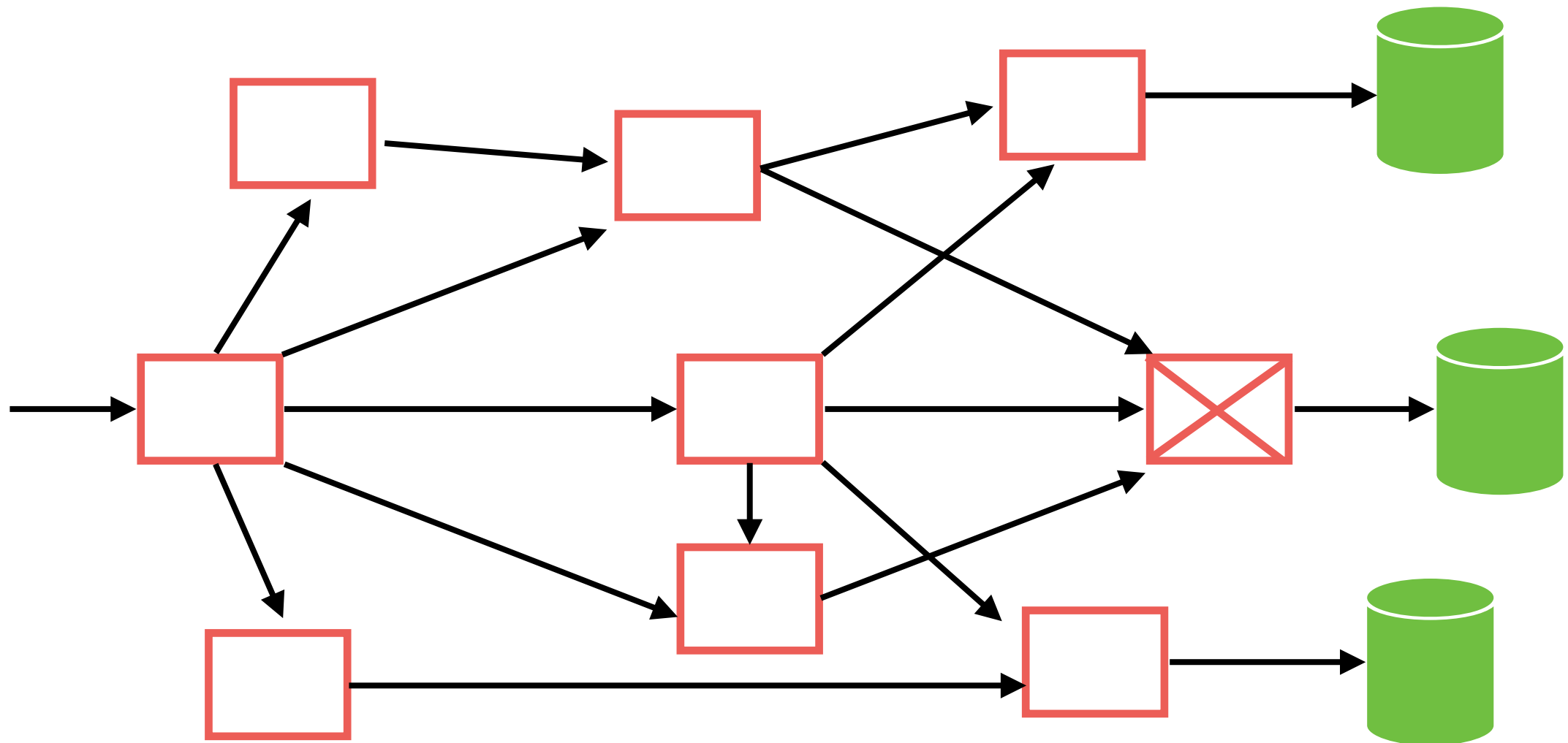
Caching data



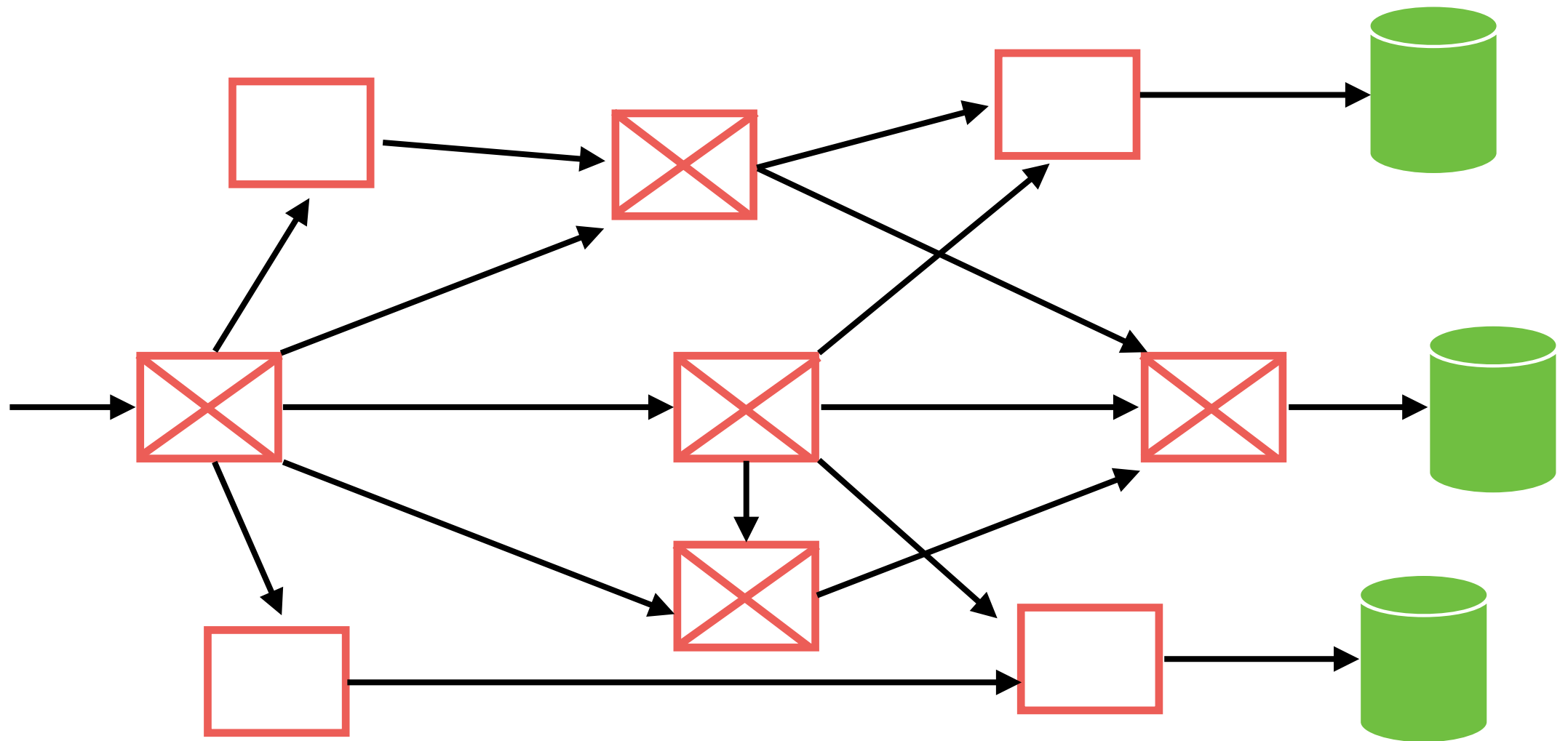
Working with Caching



Failure !!



Cascading Failure !!



Solutions for Resilience ?

Timeout

Retry

Rate Limit

Queue messaging

Circuit breaker

Bulkhead

Auto-scale

<https://learn.microsoft.com/en-us/azure/well-architected/resiliency/reliability-patterns>



Design REST APIs



Users

Method	Path	Description
GET	/users	Get all users
GET	/users/{id}	Get user by id
POST	/users	Create new user
PUT	/users/{id}	Update user
DELETE	/users/{id}	Delete user by id



HTTP status codes

Code	Description	Example
1xx	Informational responses	100 = Continue 101 Switching protocols
2xx	Success	200 = OK 201 = Created
3xx	Redirection	300 = Multiple choices 301 = Moved permanently
4xx	Client errors	400 = Bad request 403 = Forbidden 404 = Not found
5xx	Server errors	500 = Internal server error 502 = Bad gateway 503 = Service unavailable



REST APIs Best Practices



REST APIs Best Practices

Secure-by-default
Idempotent design
Consistency
Pagination
Documentation



API Documentation

pet Everything about your Pets		
POST	/pet/{petId}/uploadImage uploads an image	🔒 ▼
POST	/pet Add a new pet to the store	🔒 ▼
PUT	/pet Update an existing pet	🔒 ▼
GET	/pet/findByStatus Finds Pets by status	🔒 ▼
GET	/pet/{petId} Find pet by ID	🔒 ▼
POST	/pet/{petId} Updates a pet in the store with form data	🔒 ▼
DELETE	/pet/{petId} Deletes a pet	🔒 ▼

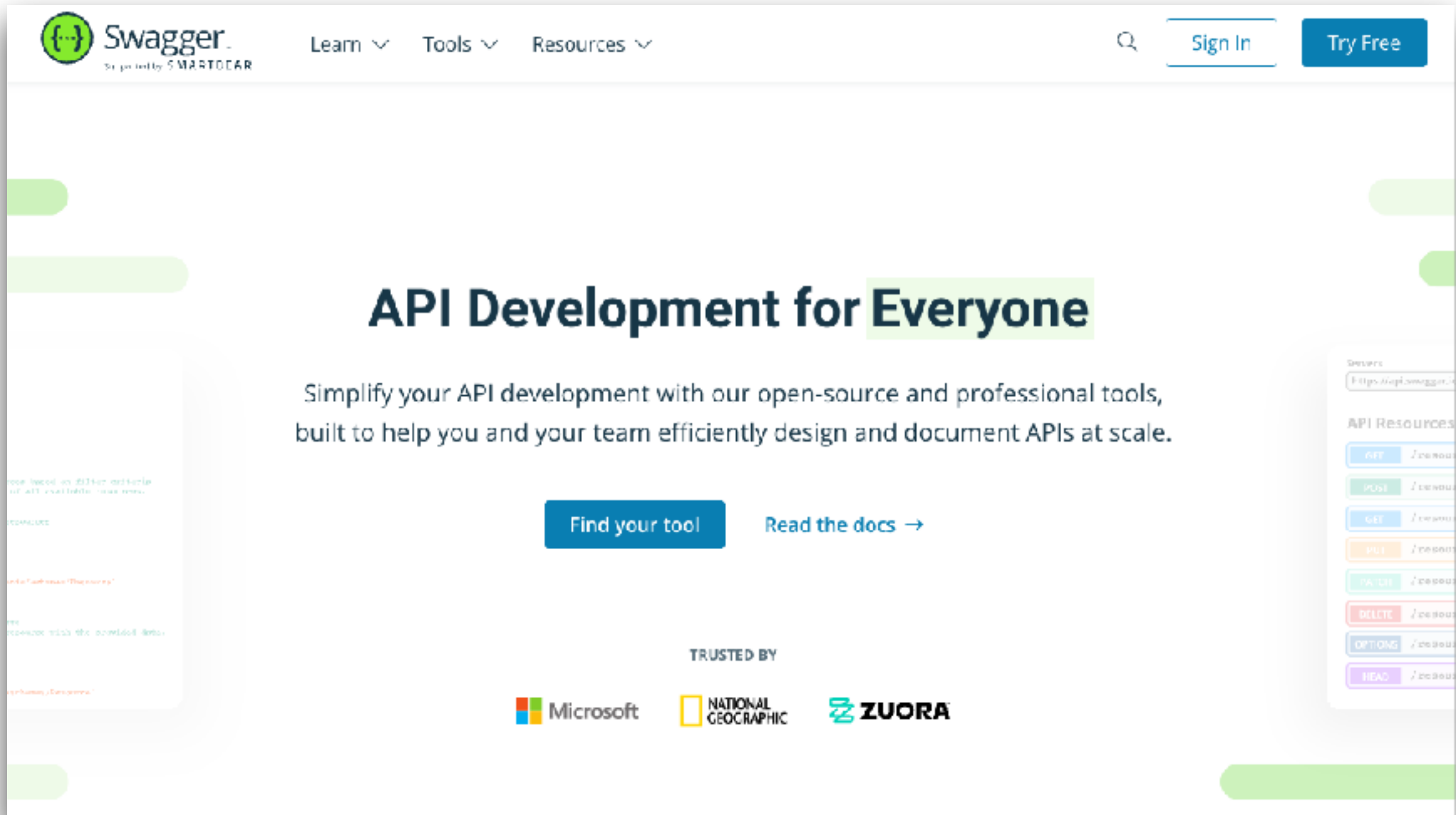
<https://swagger.io/>



API Documentation



Swagger/ Open API



<https://swagger.io/>



Swagger + Spring Boot

Spring Doc



<https://springdoc.org/>



Postman



<https://www.postman.com/downloads/>



Response Status

404 = Not Found

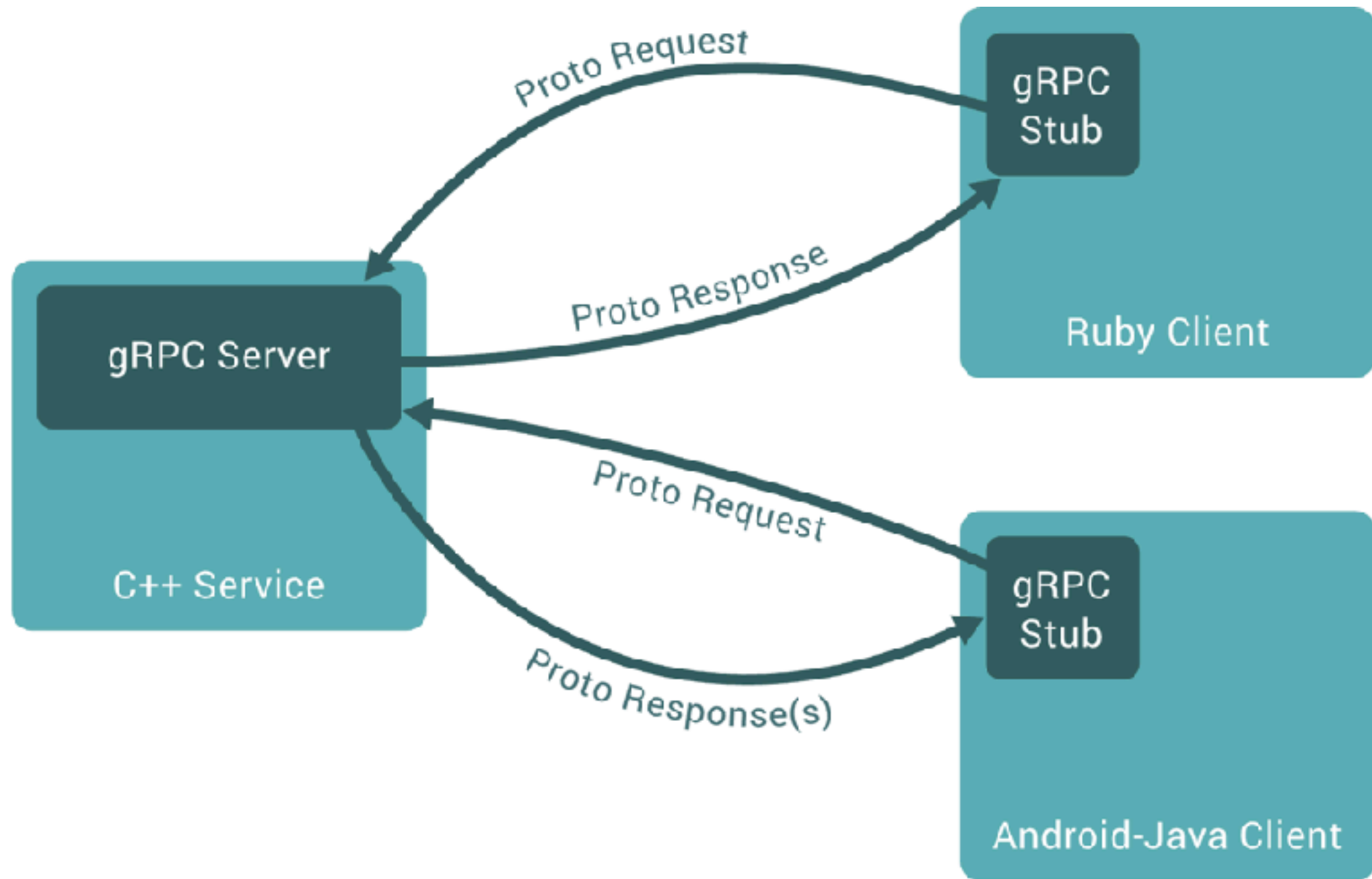
Status	Description
400	Request body doesn't meet API spec
401	Authentication/Authorization fail
403	User can't perform the operation
404	Resource does not exist
405	Unsupported operation
500	Error on server



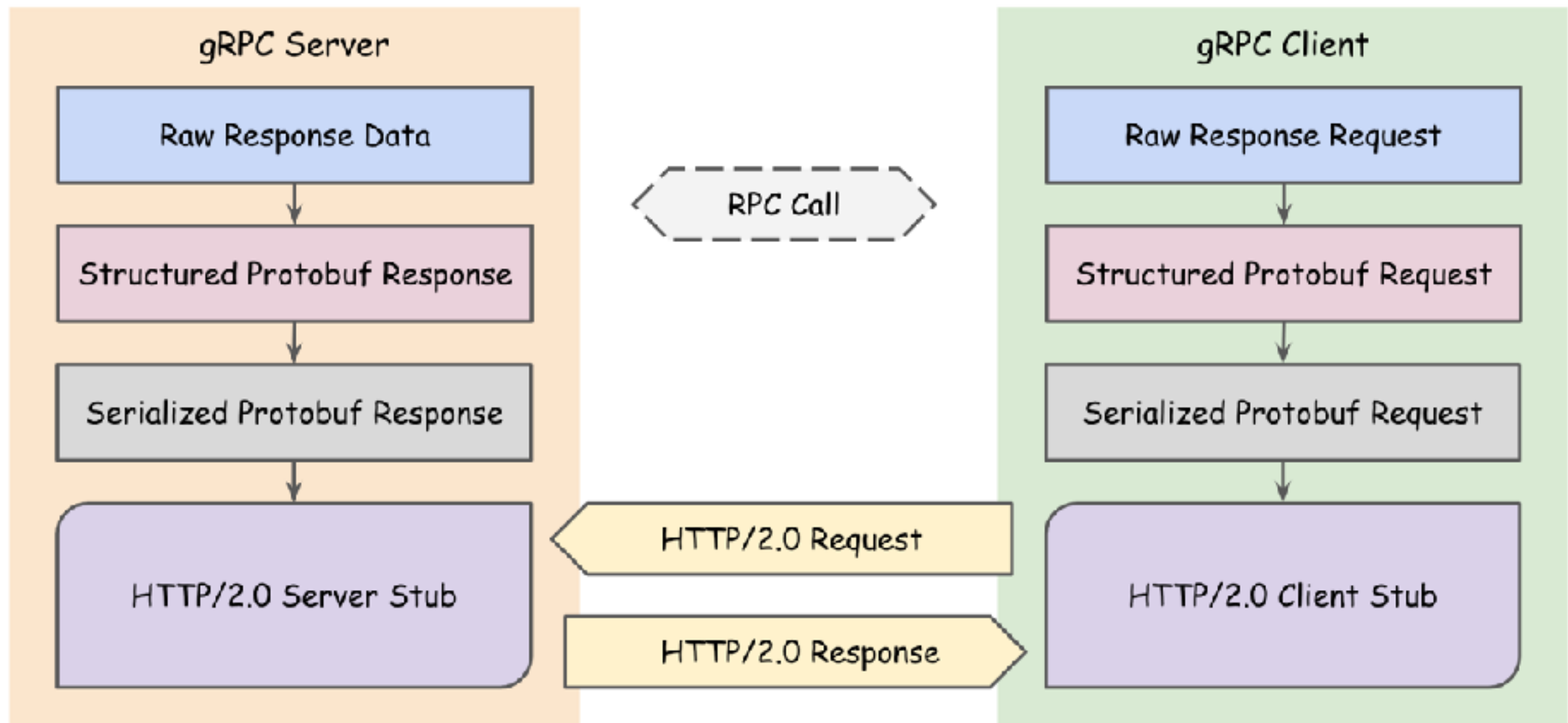
gRPC



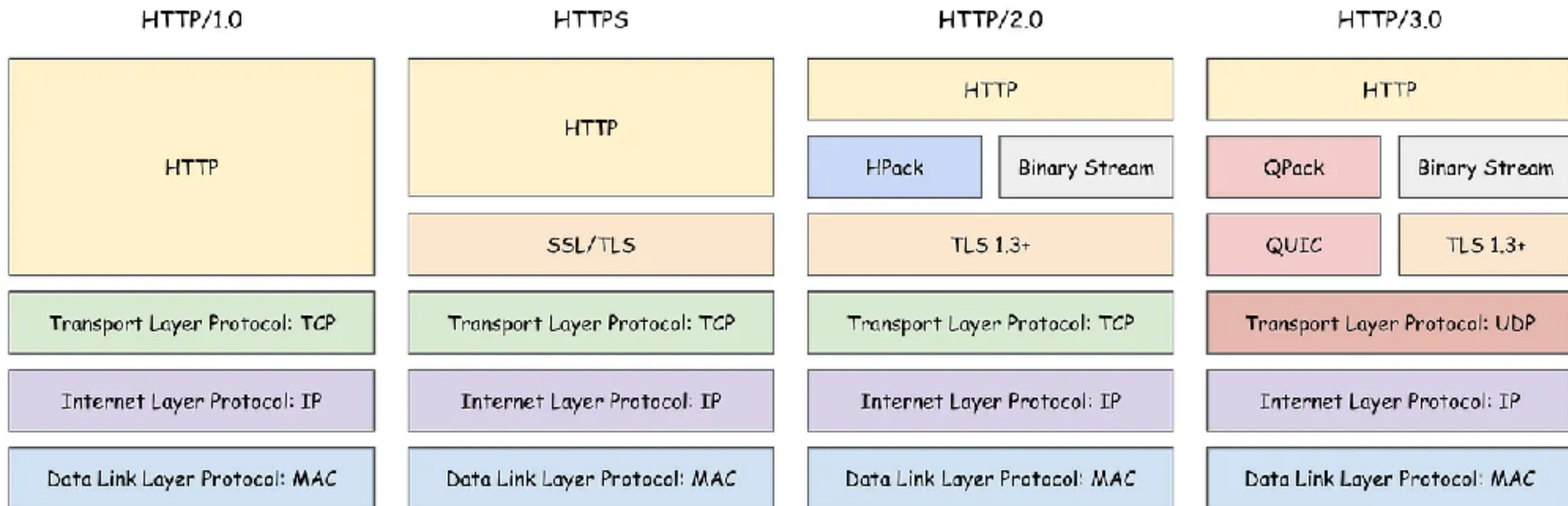
gRPC Workflow



gRPC Architecture



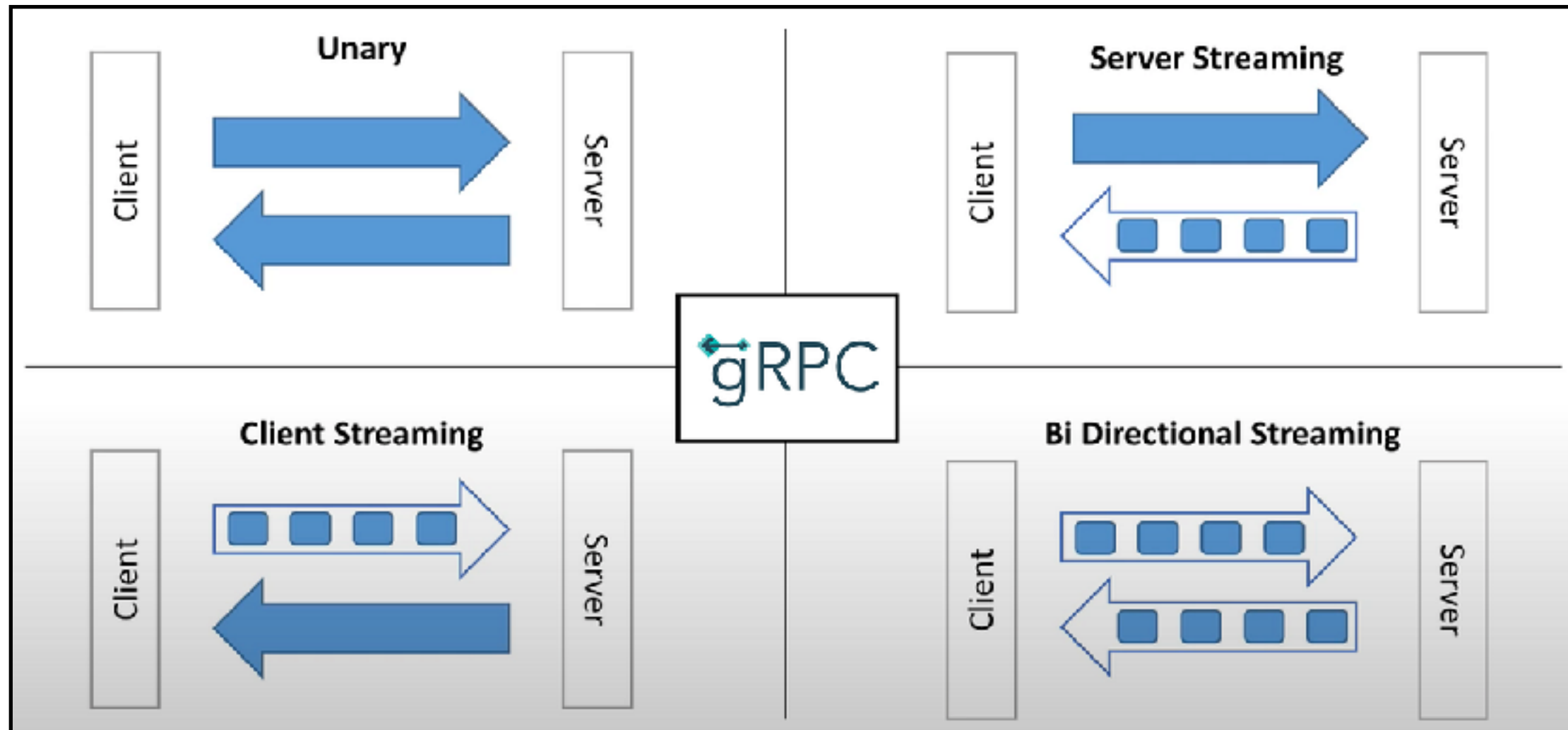
Evolution of HTTP



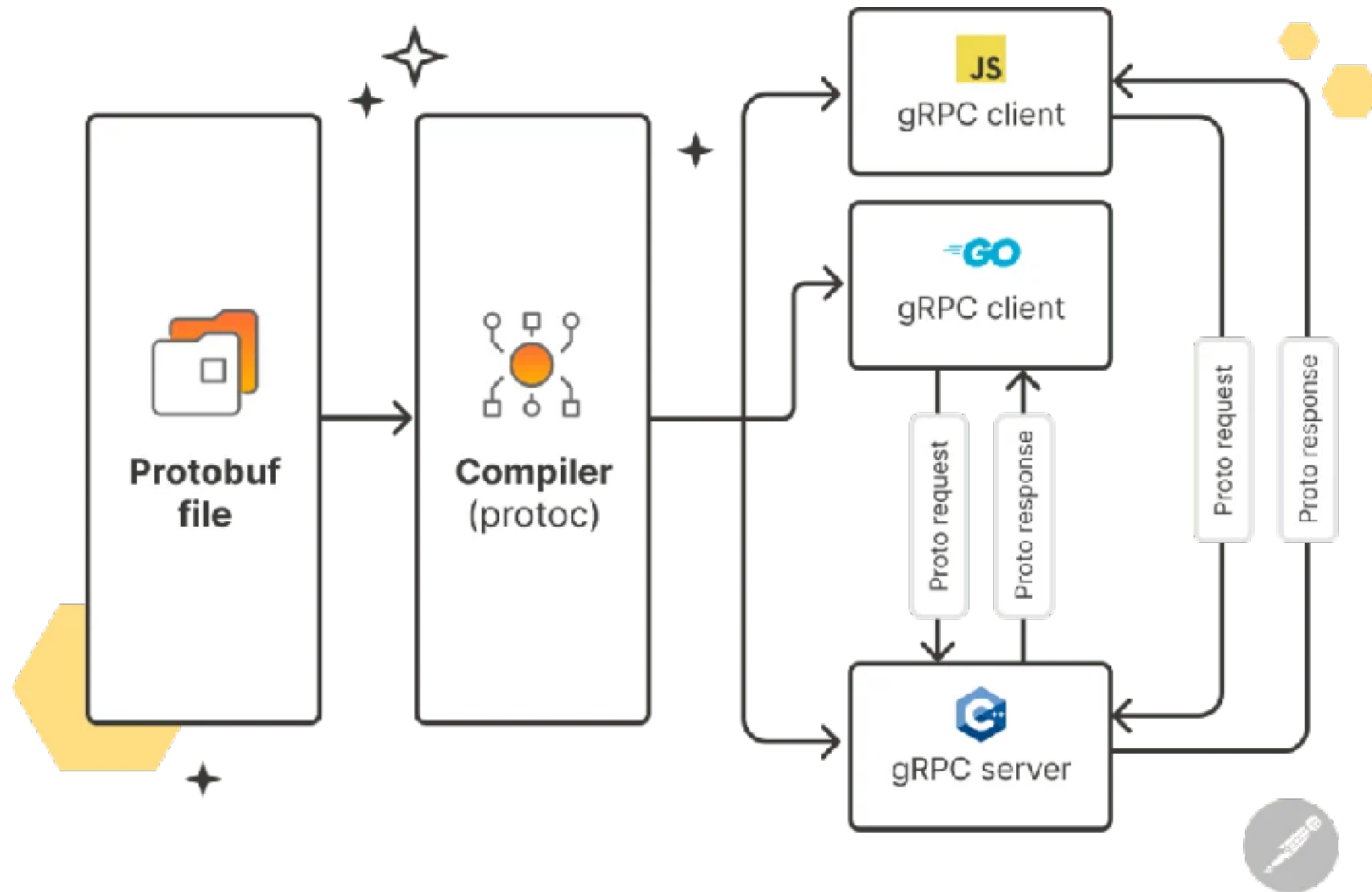
<https://medium.com/@lchang1994/deep-dive-grpc-protobuf-http-2-0-74e6295f1d38>



gRPC via HTTP/2



gRPC Workflow



Important Quality Service



Important Quality Service

Security
Observability
Configurability



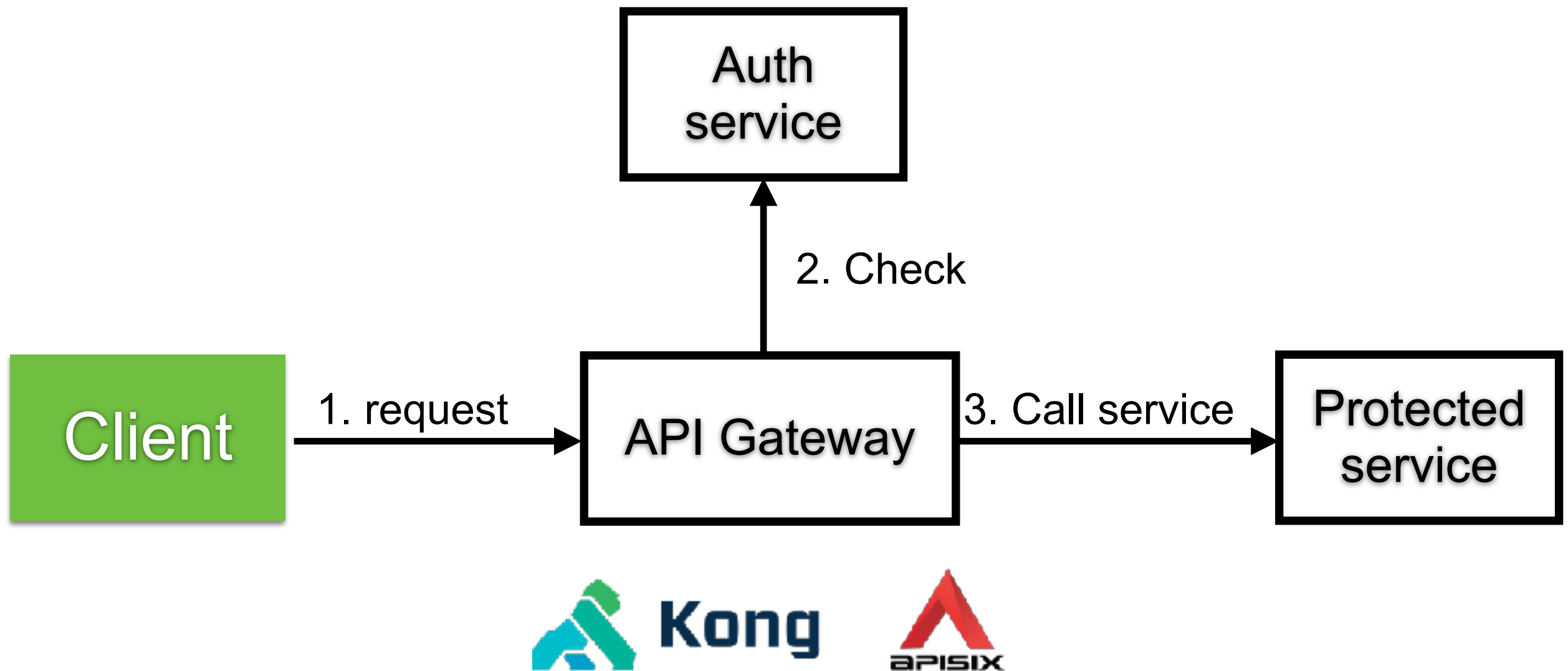
Secure Service ?

API Gateway

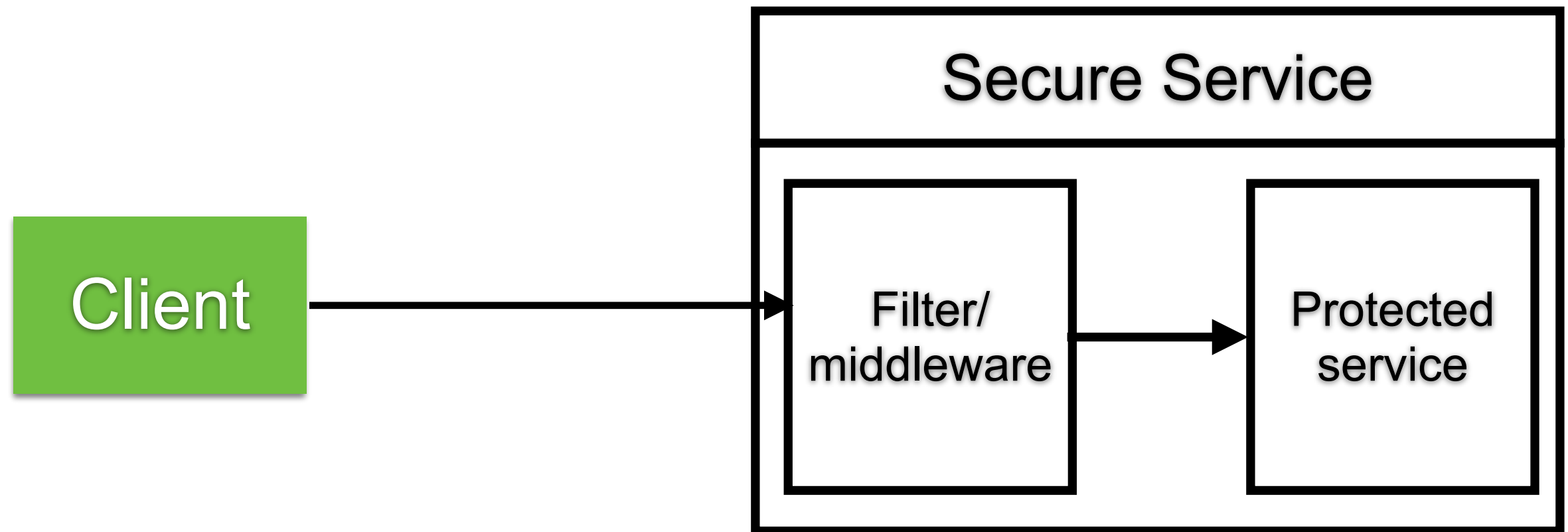
Integrate in your system/service



API Gateway



Within service

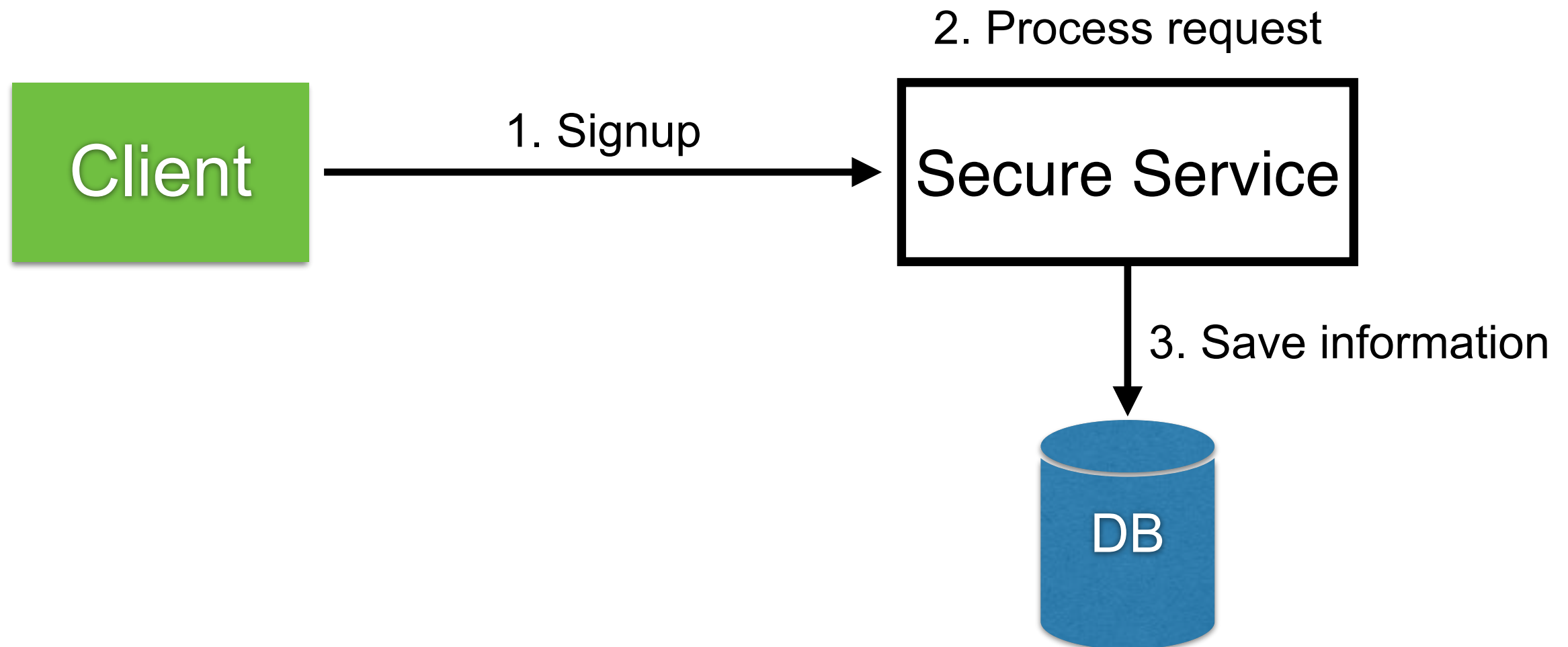


<https://github.com/up1/demo-spring-security>

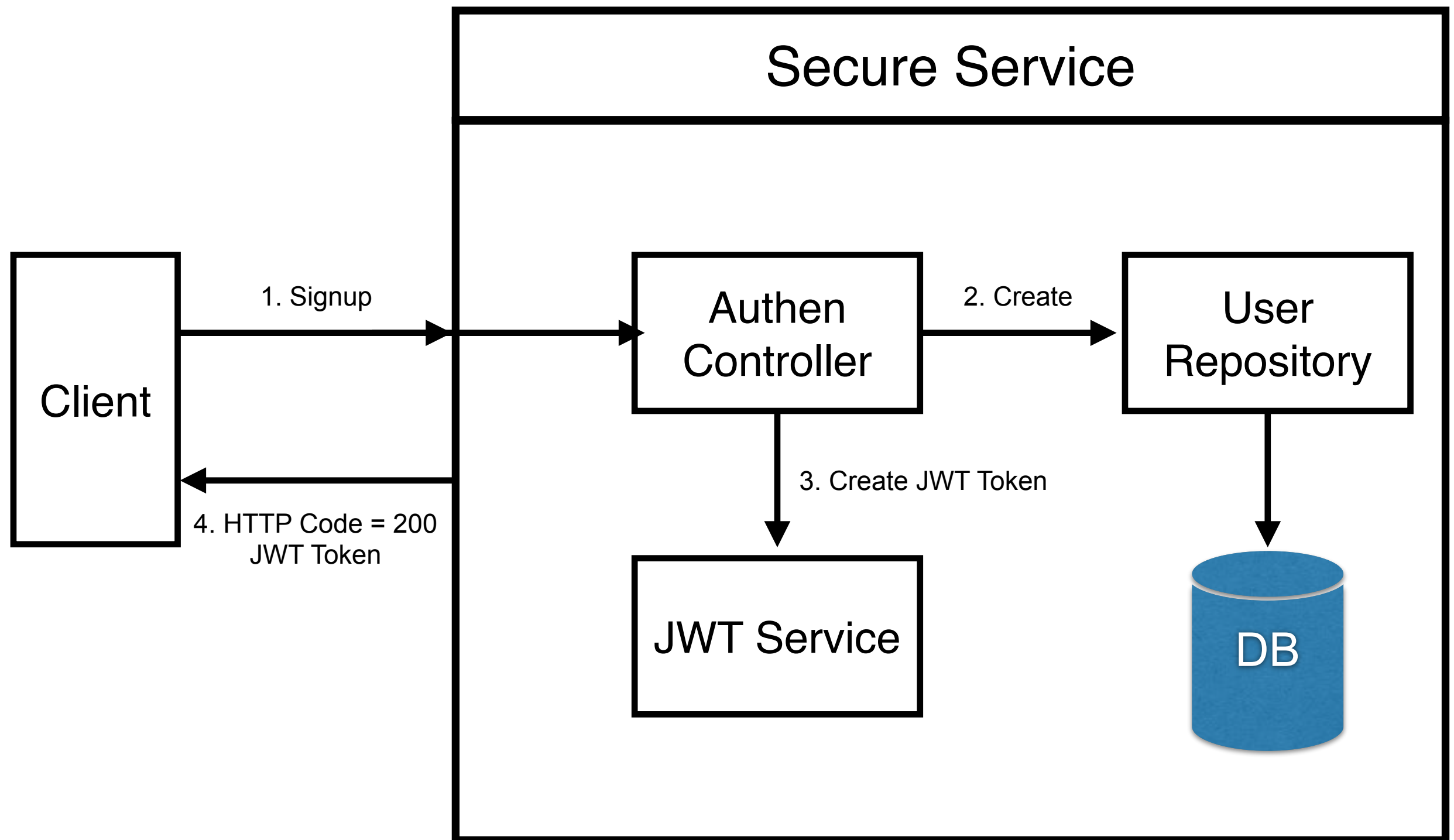


Simple Flow of Secure Service

Step 1 :: Signup process

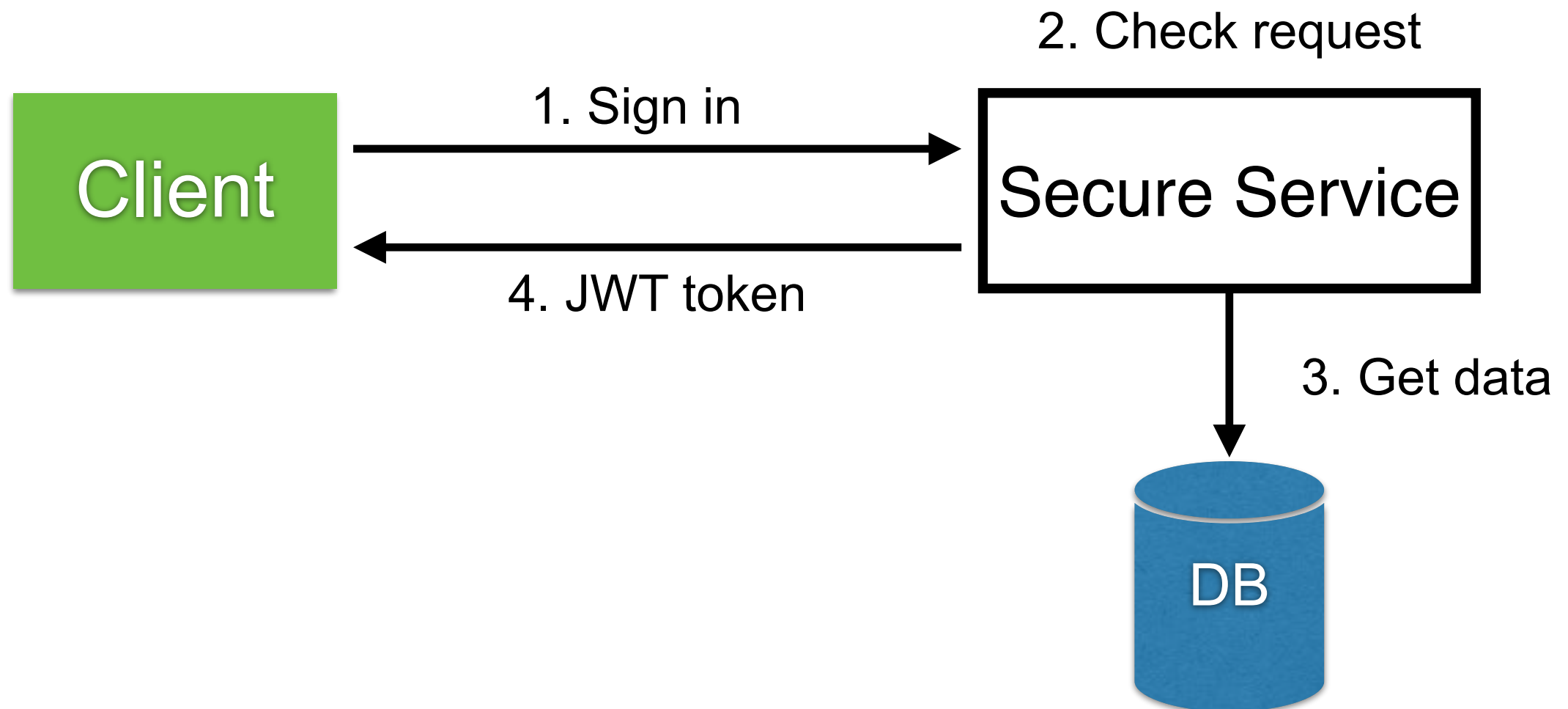


Flow diagram of Signup

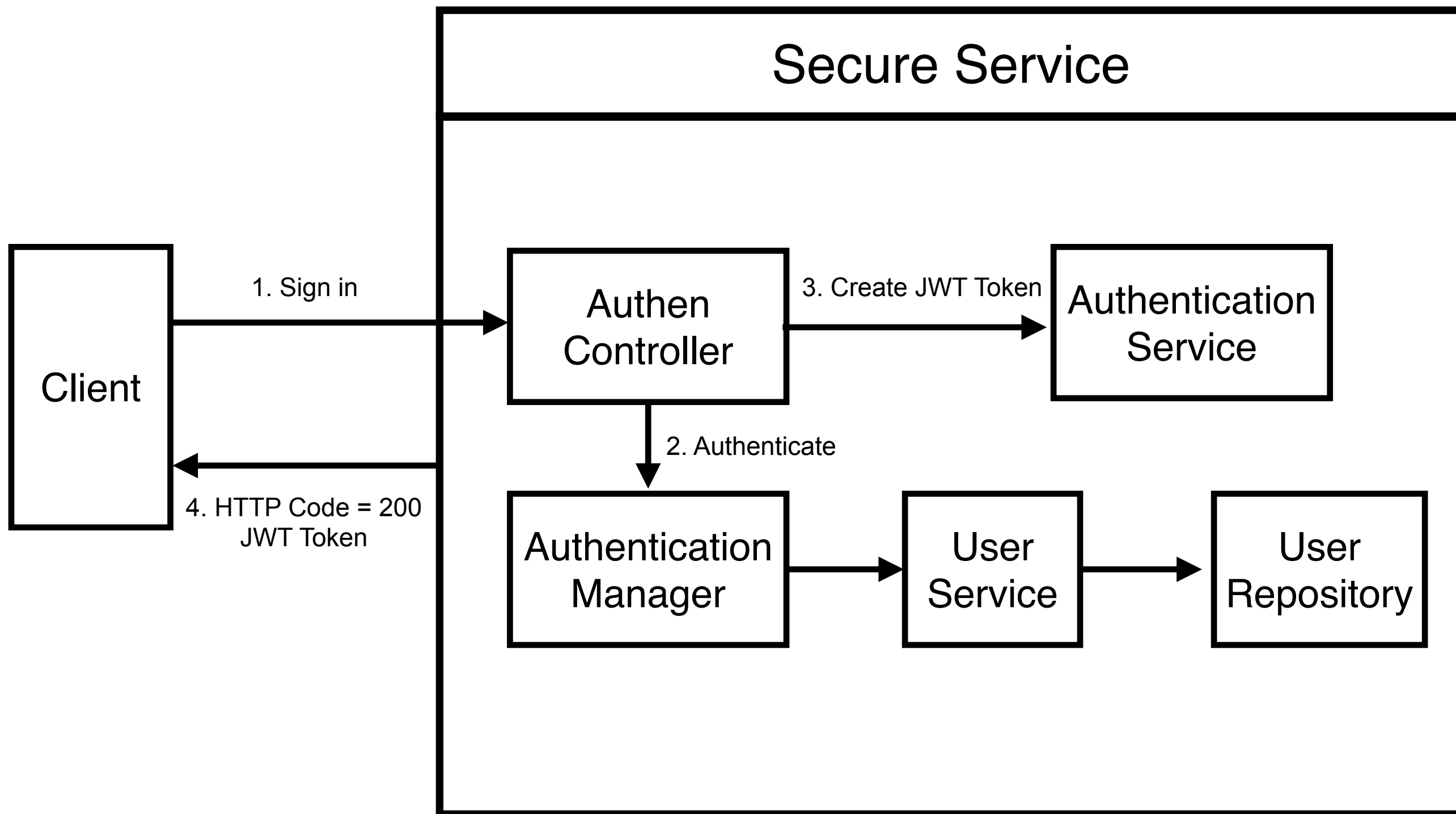


Simple Flow of Secure Service

Step 2 :: Sign in process

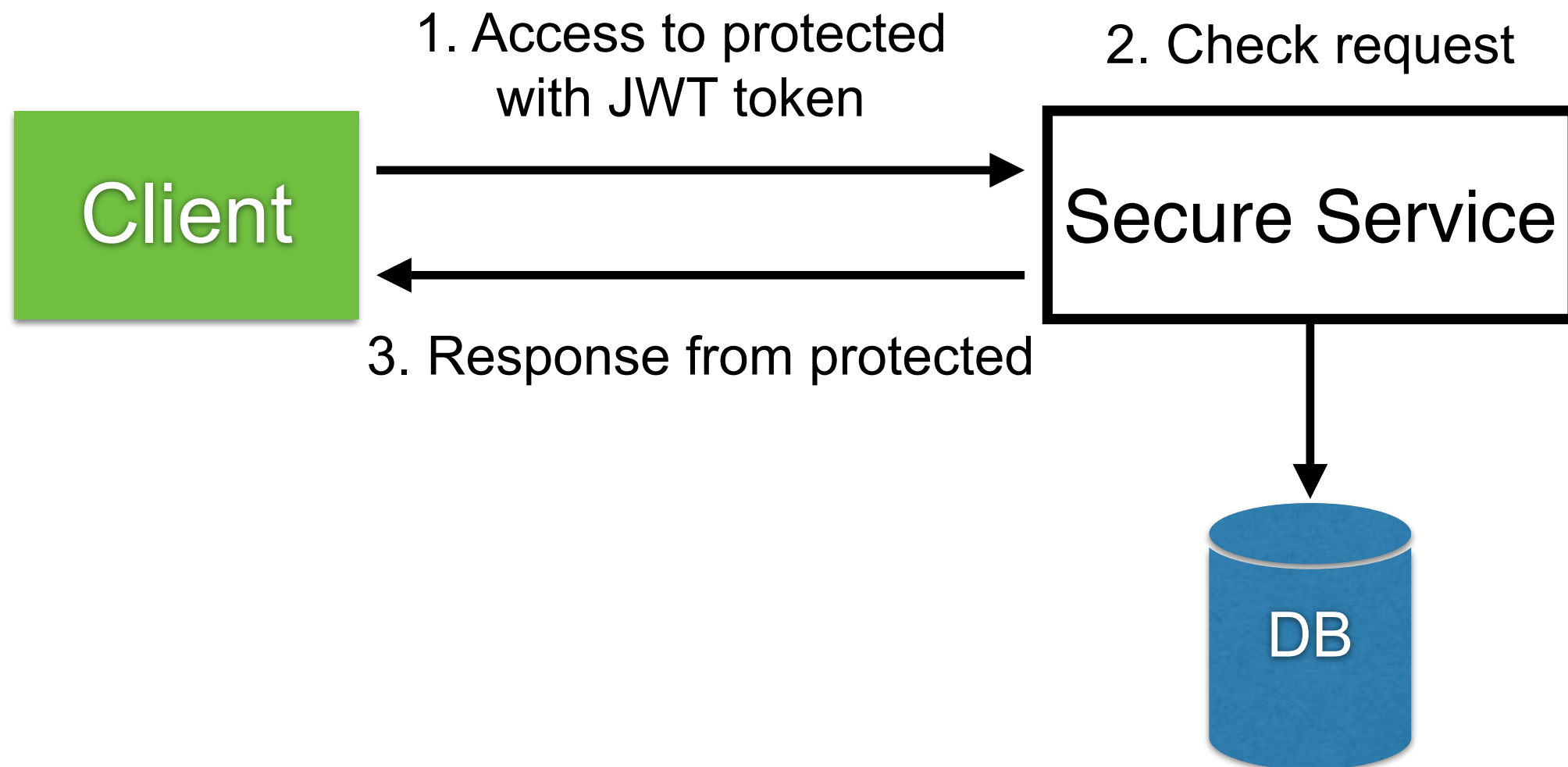


Flow diagram of Sign in

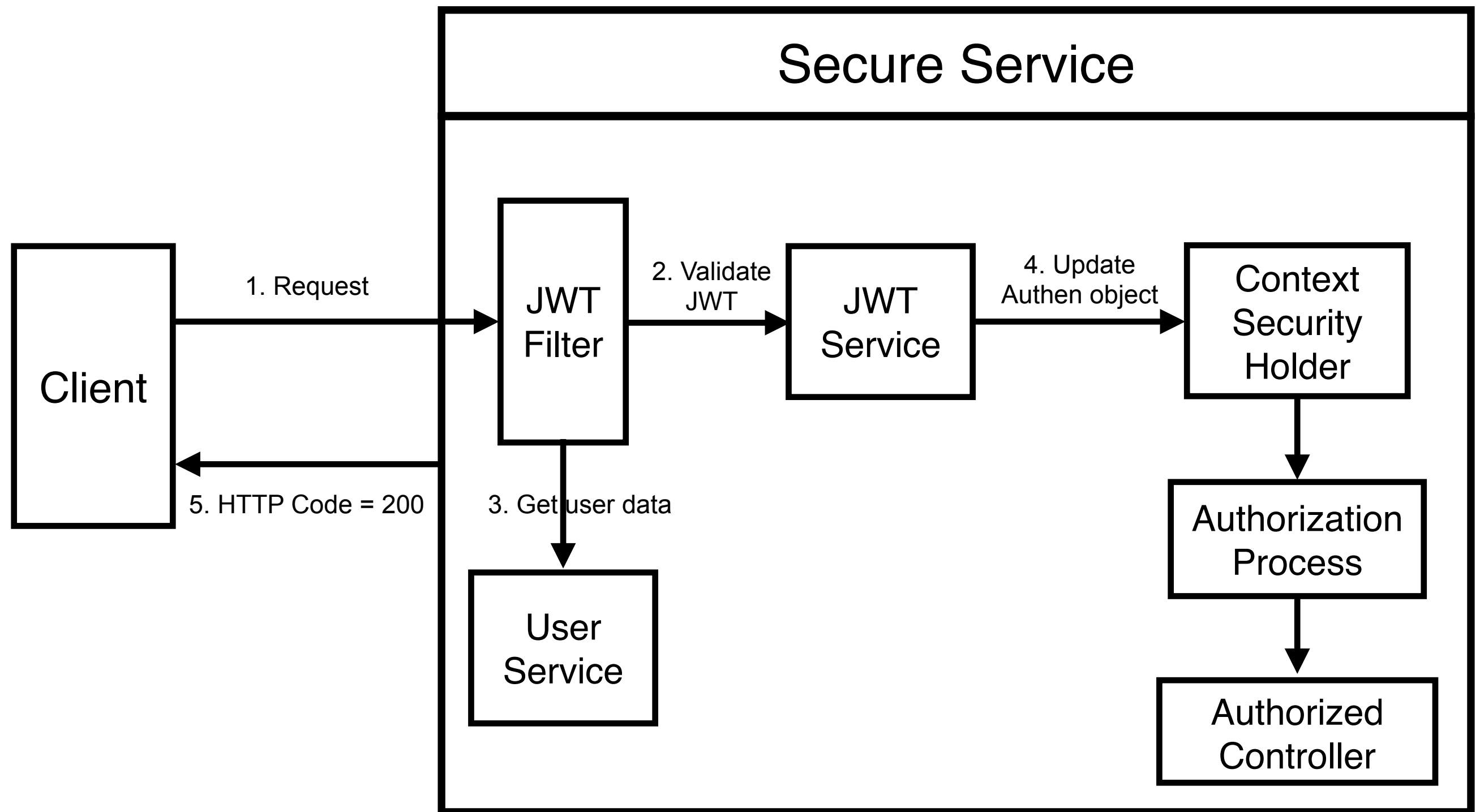


Simple Flow of Secure Service

Step 3 :: Access to protected services



Flow diagram of call service



JSON Web Tokens (JWT)

Encoded

PASTE A TOKEN HERE

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

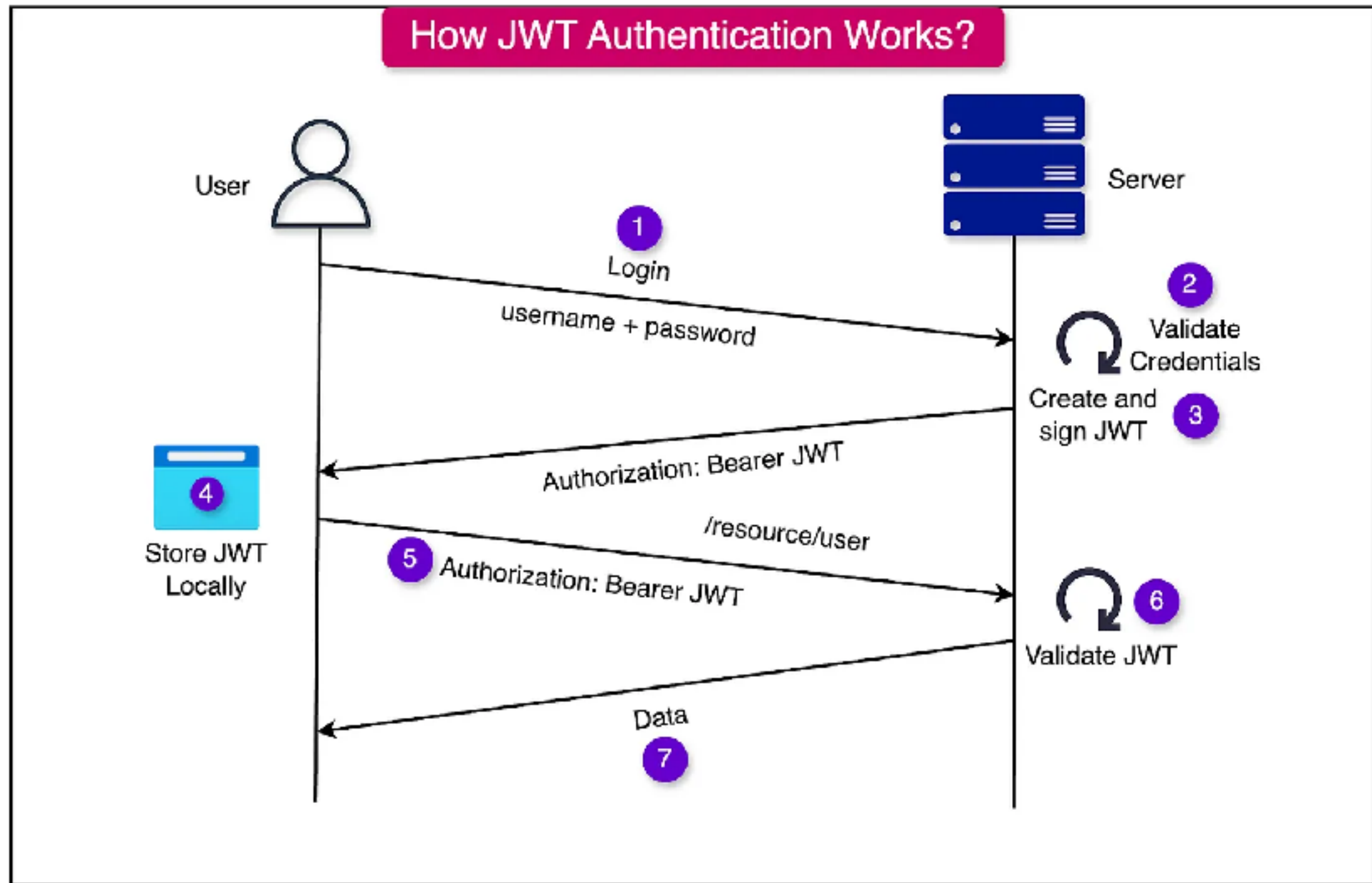
Signature Verified

SHARE JWT

<https://jwt.io/>



Working with JWT



<https://blog.bytebytego.com/p/mastering-modern-authentication-cookies>



OWASP TOP 10 API

<https://owasp.org/API-Security/editions/2023/en/0x11-t10/>



2019

- 1 Broken Object Level Authorization
- 2 Broken User Authentication
- 3 Excessive Data Exposure
- 4 Lack of Resources & Rate Limiting
- 5 Broken Function Level Authorization
- 6 Mass Assignment
- 7 Security Misconfiguration
- 8 Injection
- 9 Improper Assets Management
- 10 Insufficient Logging & Monitoring

2023

- Broken Object Level Authorization 1
- Broken Authentication 2
- Broken Object Property Level Authorization 3
- Unrestricted Resource Consumption 4
- Broken Function Level Authorization 5
- Server Side Request Forgery 6
- Security Misconfiguration 7
- Lack of Protection from Automated Threats 8
- Improper Inventory Management 9
- Unsafe Consumption of APIs 10



Broken Object Level Authorization (BOLA)

Occurs when API endpoint fails to adequately
verify

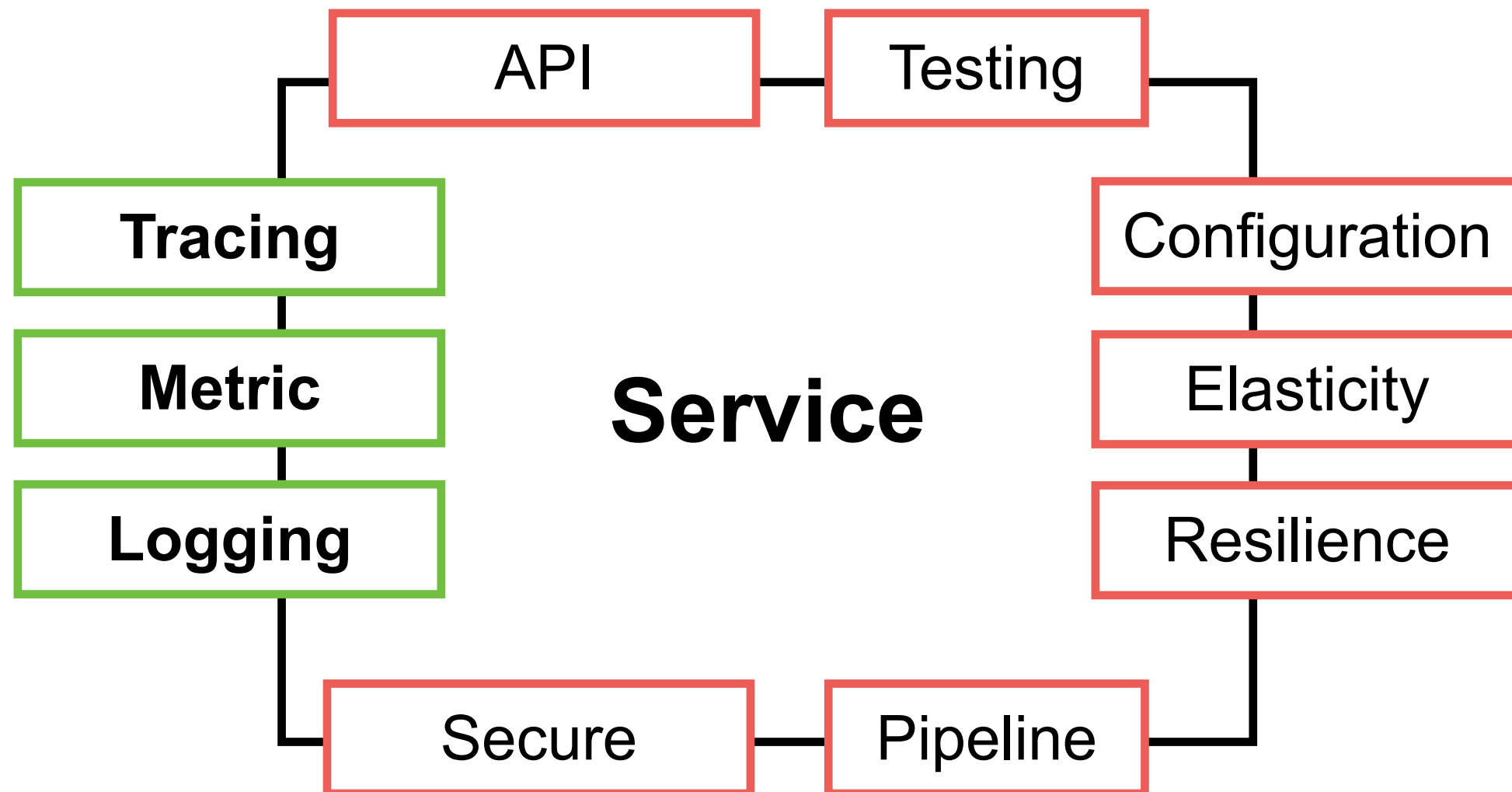
BOLA happens when authentication succeeds but
object-level authorization is missing



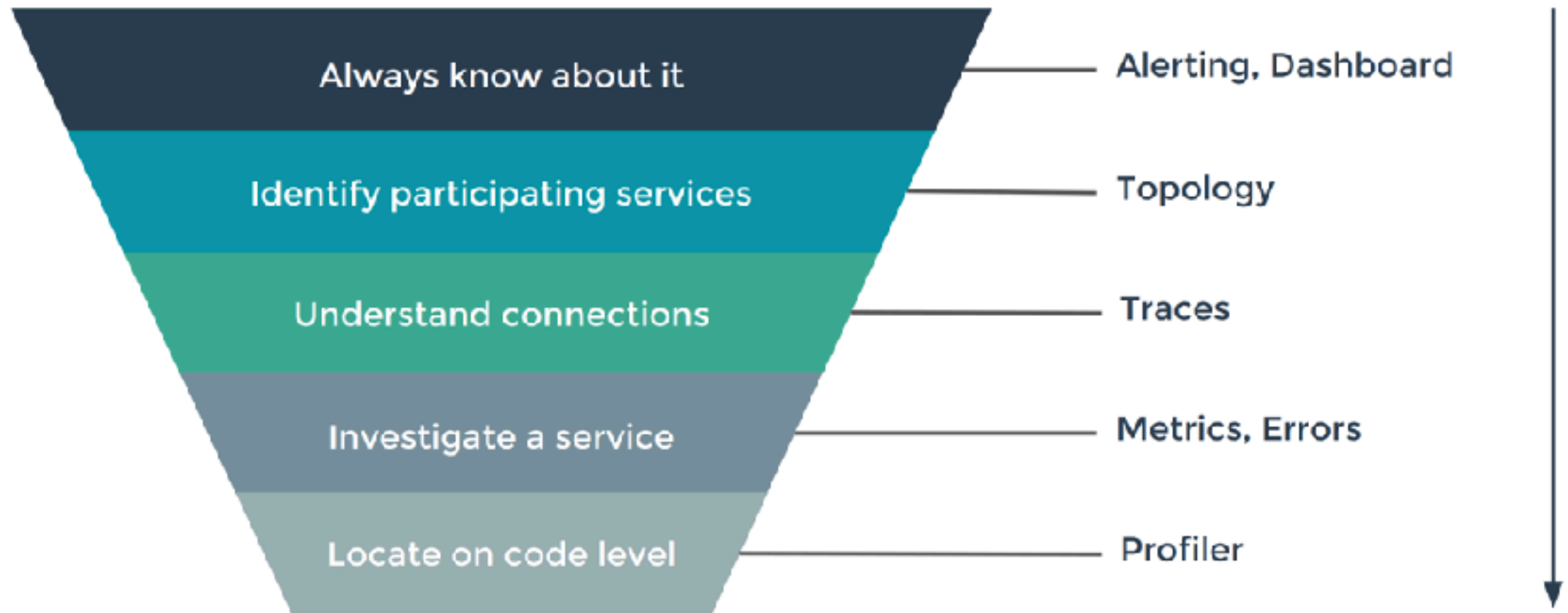
Observable Service



Properties of Service



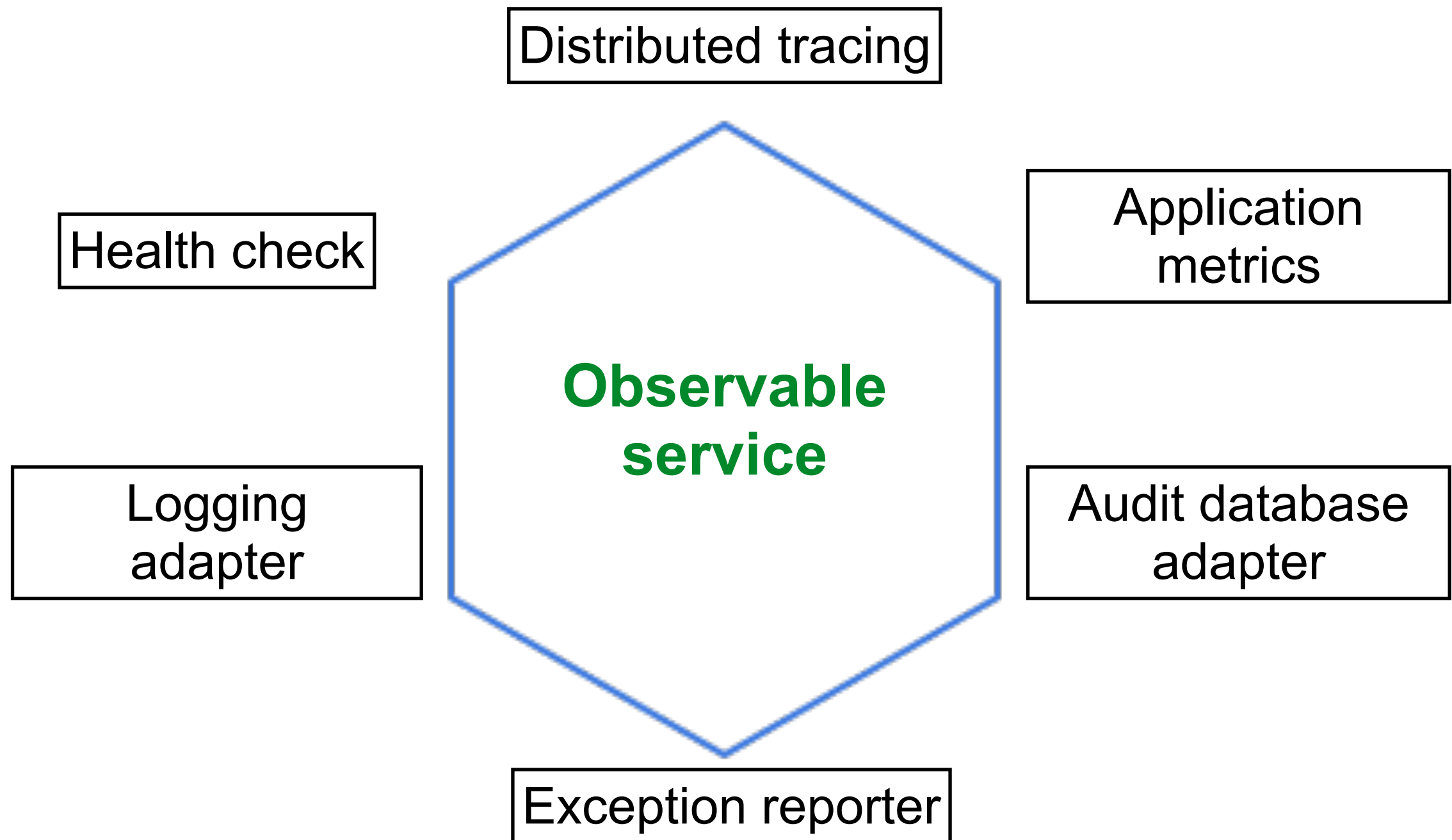
How to find an issue ?



Observability vs Monitoring

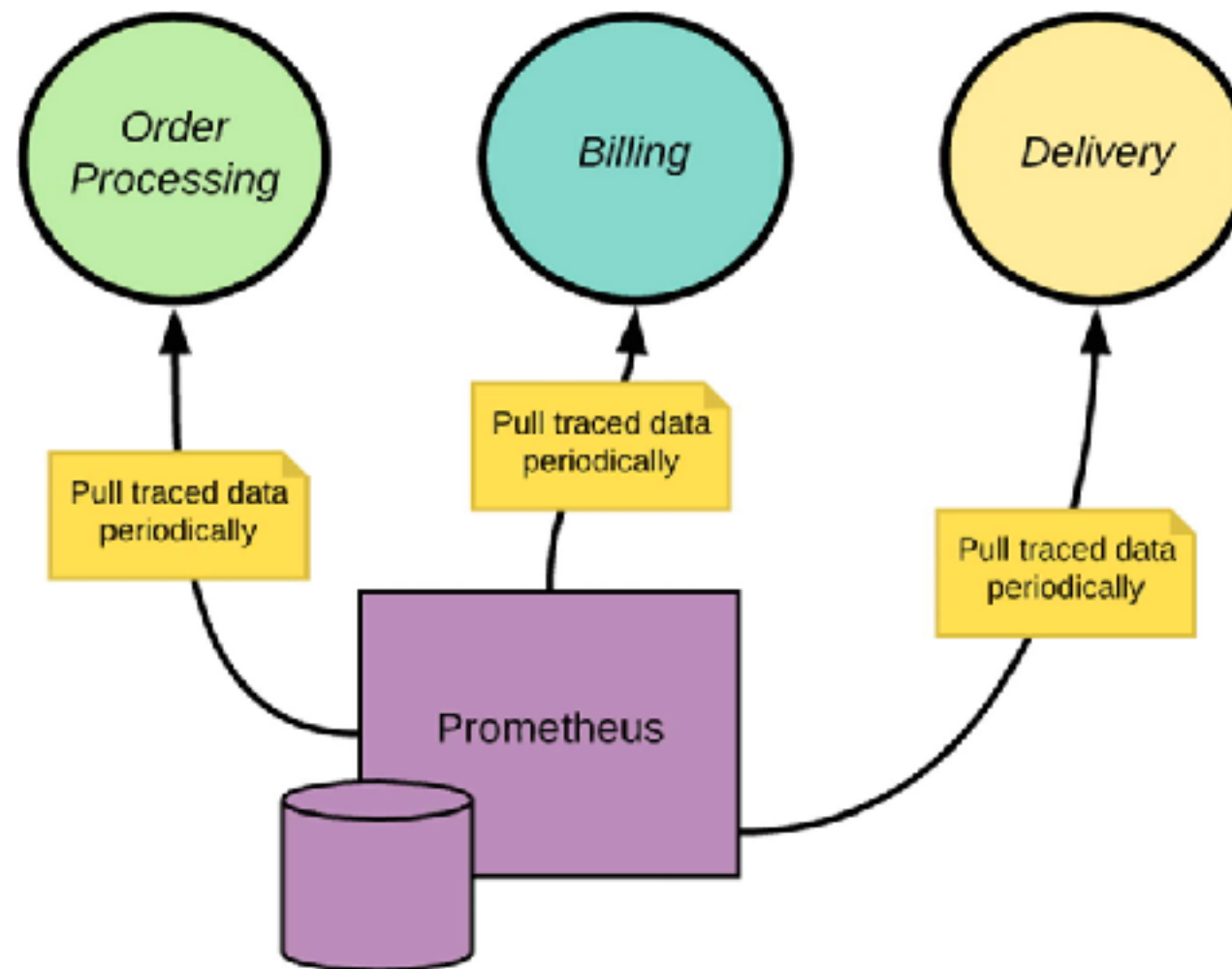


Observable services



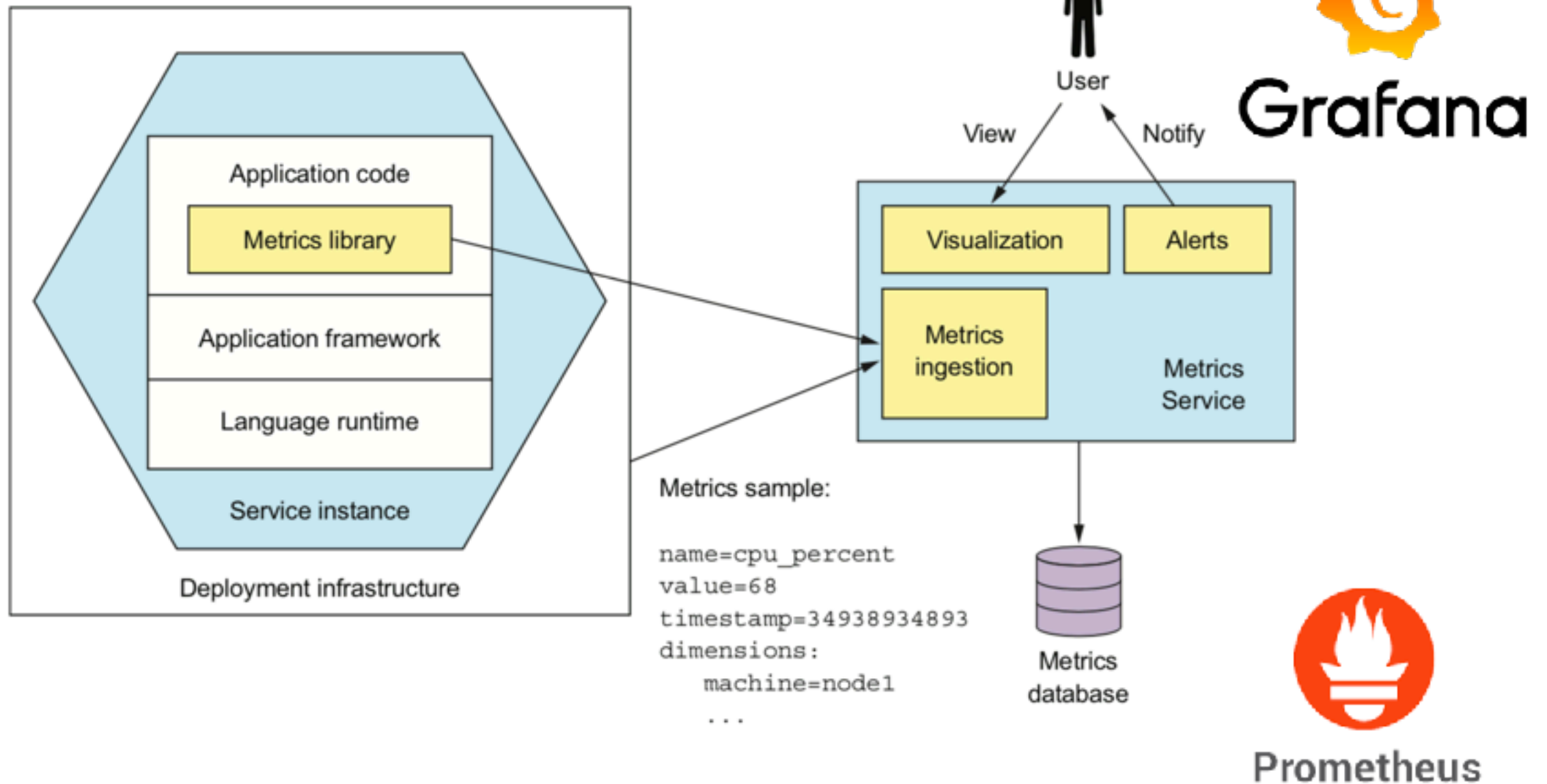
Application metrics

Services maintain metrics and expose to metric server (counters, gauges)

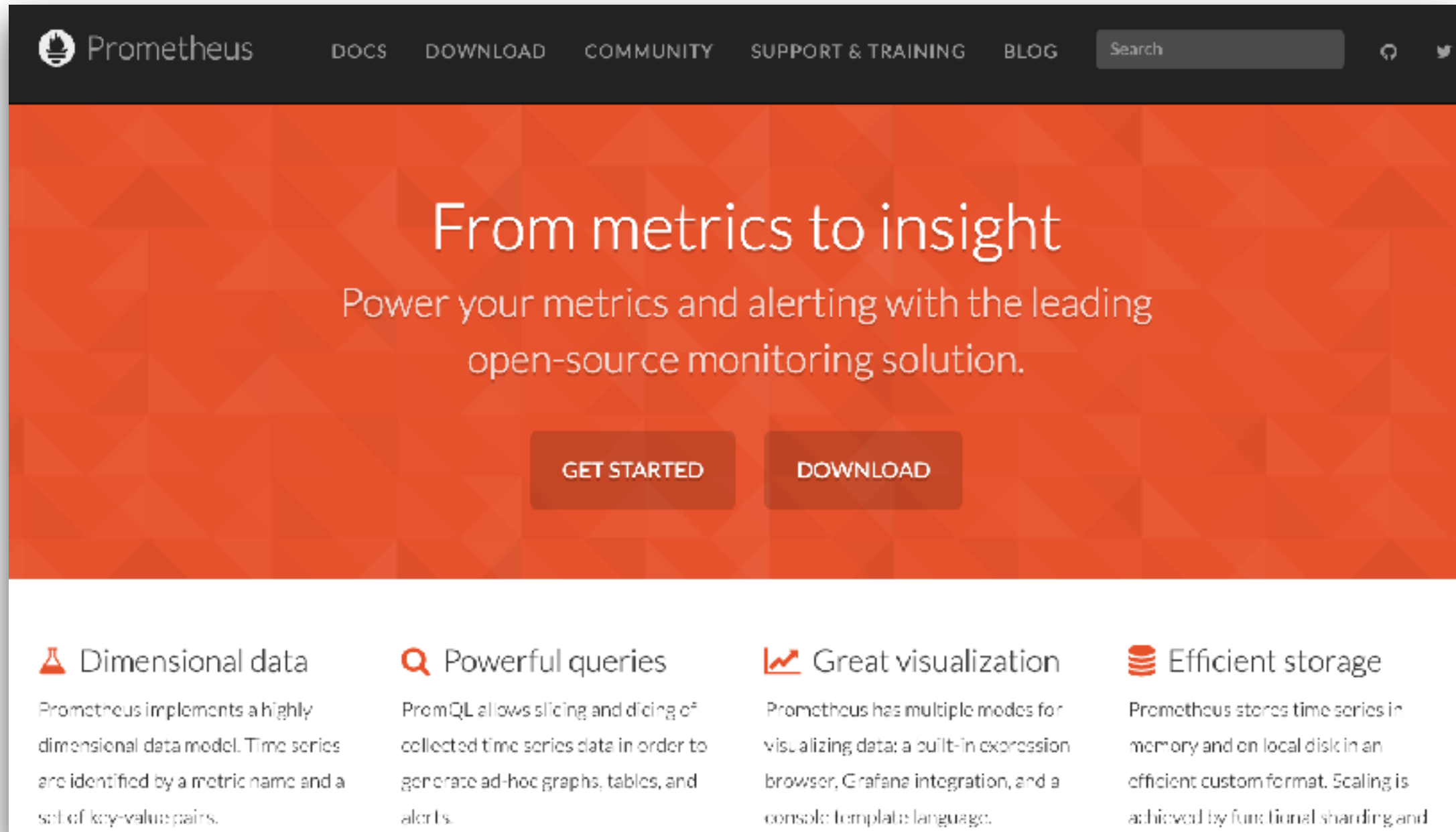


Application metrics tools

Spring Boot Actuator



Prometheus



The screenshot shows the Prometheus website homepage. At the top is a dark navigation bar with the Prometheus logo and links for DOCS, DOWNLOAD, COMMUNITY, SUPPORT & TRAINING, and BLOG. A search bar is on the right. The main hero section has an orange background with the text 'From metrics to insight' and 'Power your metrics and alerting with the leading open-source monitoring solution.' Below this are two buttons: 'GET STARTED' and 'DOWNLOAD'. The bottom section features four columns with icons and descriptions of Prometheus features: Dimensional data, Powerful queries, Great visualization, and Efficient storage.

Prometheus

DOCS DOWNLOAD COMMUNITY SUPPORT & TRAINING BLOG Search

From metrics to insight

Power your metrics and alerting with the leading open-source monitoring solution.

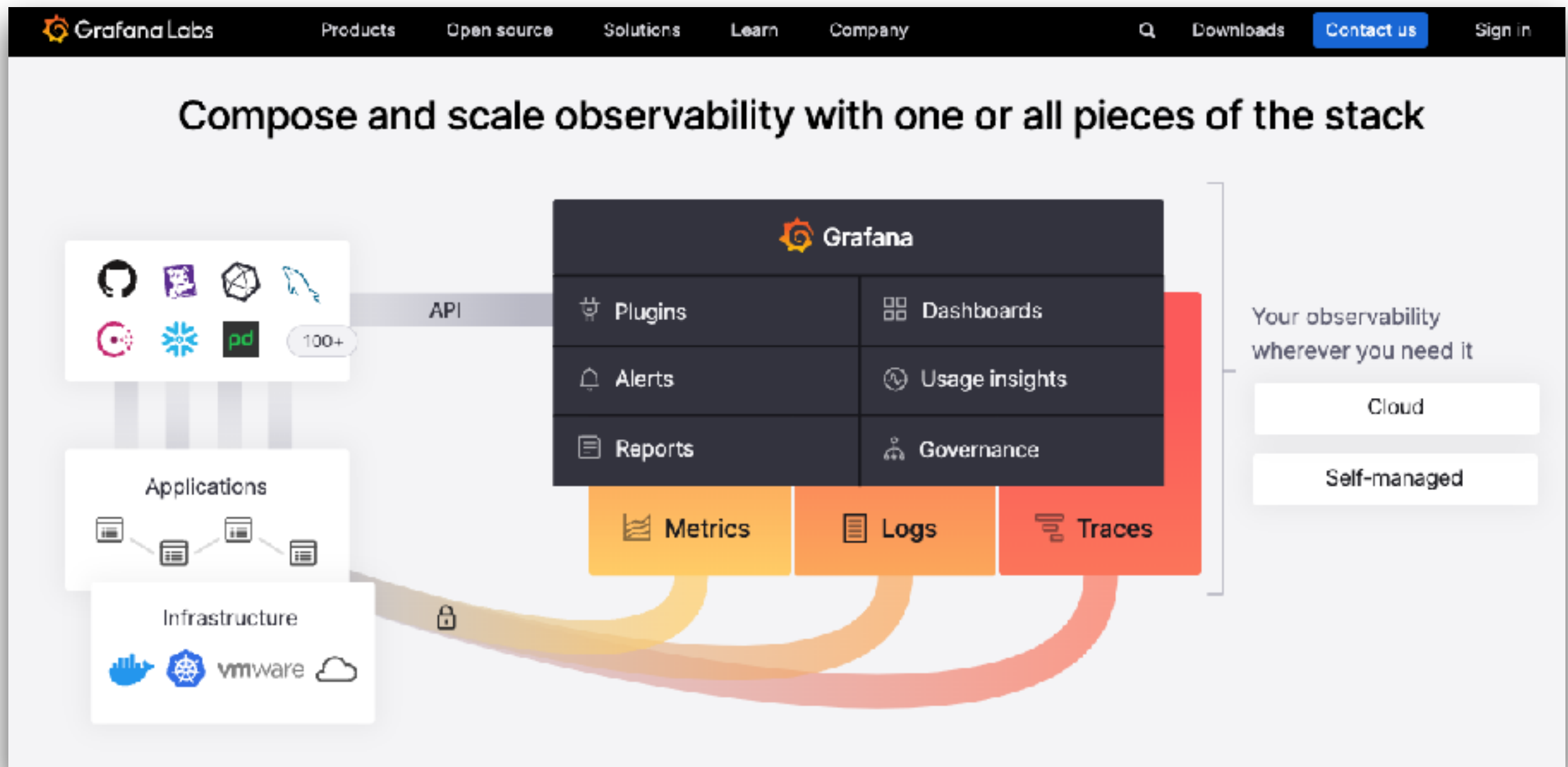
GET STARTED DOWNLOAD

-  **Dimensional data**
Prometheus implements a highly dimensional data model. Time series are identified by a metric name and a set of key-value pairs.
-  **Powerful queries**
PromQL allows slicing and dicing of collected time series data in order to generate ad-hoc graphs, tables, and alerts.
-  **Great visualization**
Prometheus has multiple modes for visualizing data: a built-in expression browser, Grafana integration, and a console template language.
-  **Efficient storage**
Prometheus stores time series in memory and on local disk in an efficient custom format. Scaling is achieved by functional sharding and

<https://prometheus.io/>



Grafana

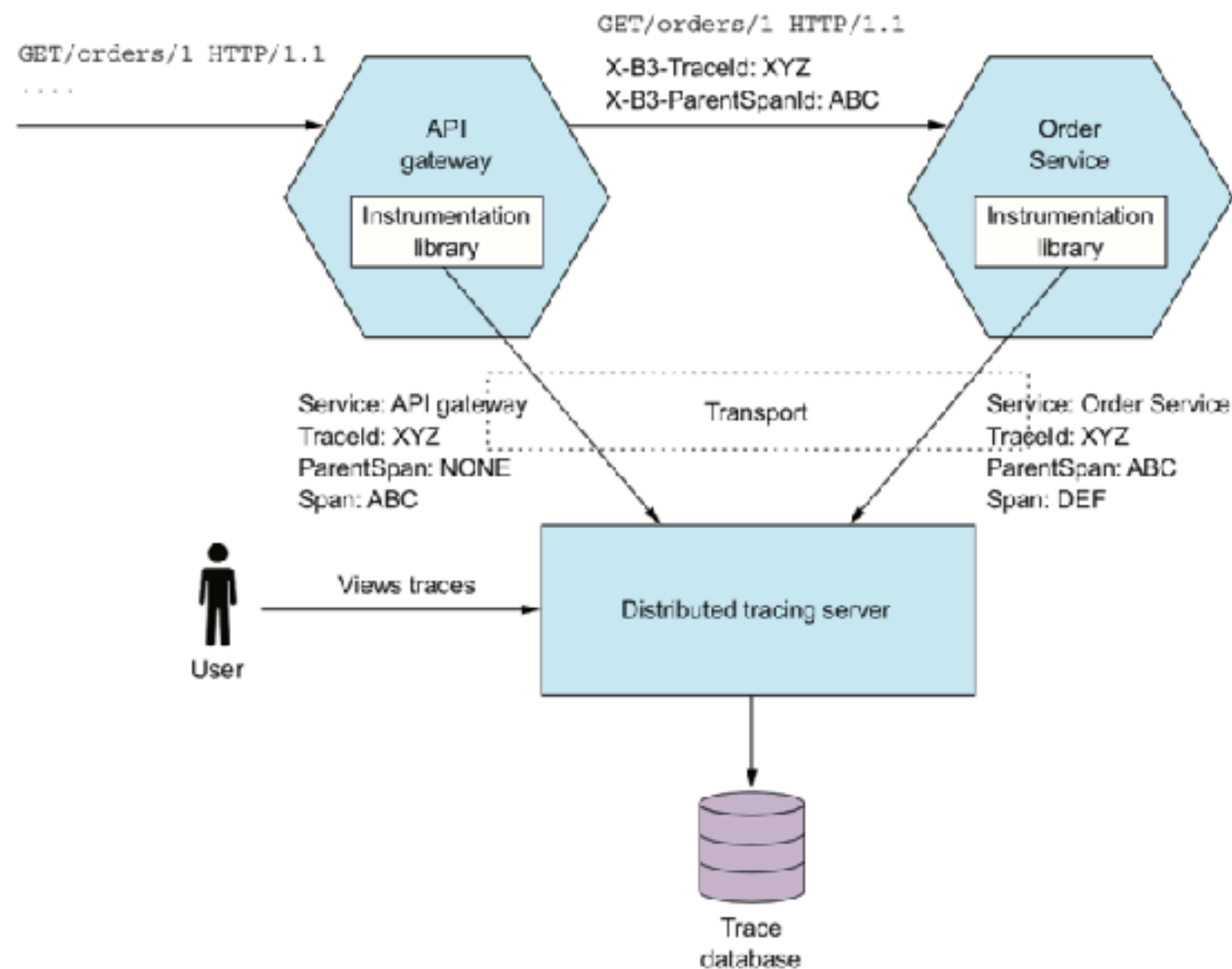


<https://grafana.com/>



Distributed tracing

Assign each external request a **unique ID** and trace requests as flow between services



Distributed tracing tools

Format standard with OpenTelemetry

Zipkin, Jaeger, Tempo, AWS X-Ray, Elastic APM



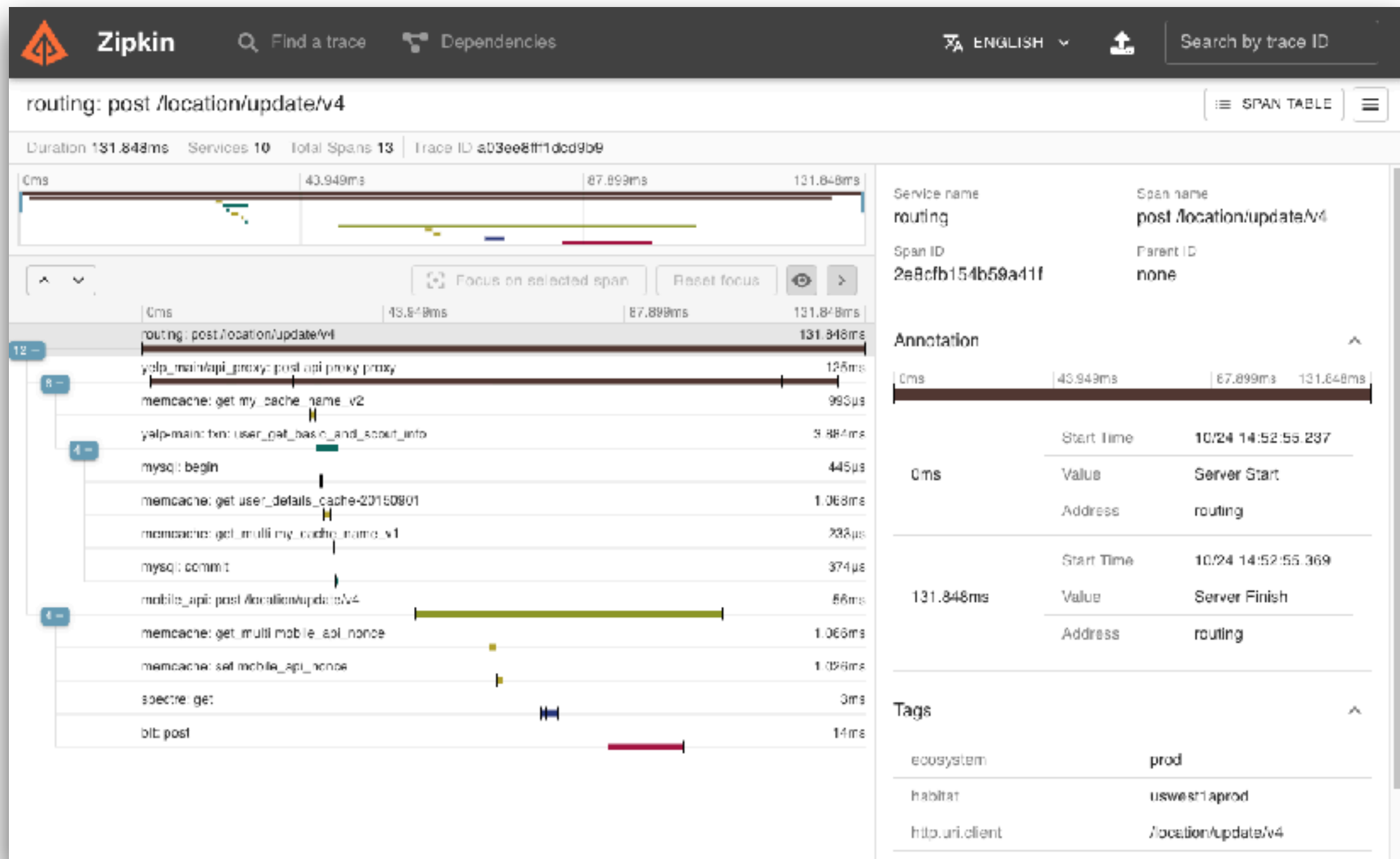
JAEGER



Grafana Tempo



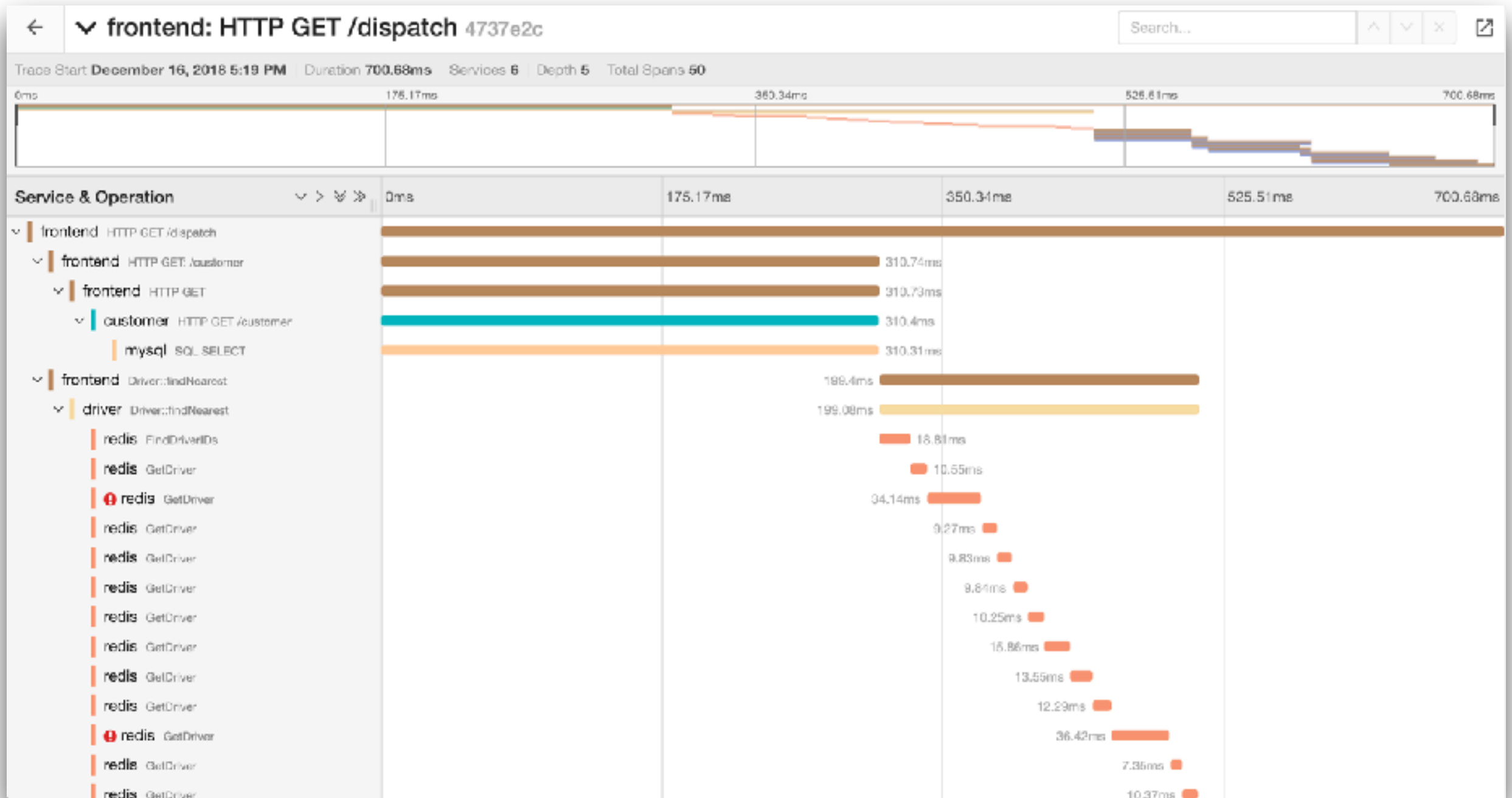
Zipkin



<https://zipkin.io/>



Jaeger

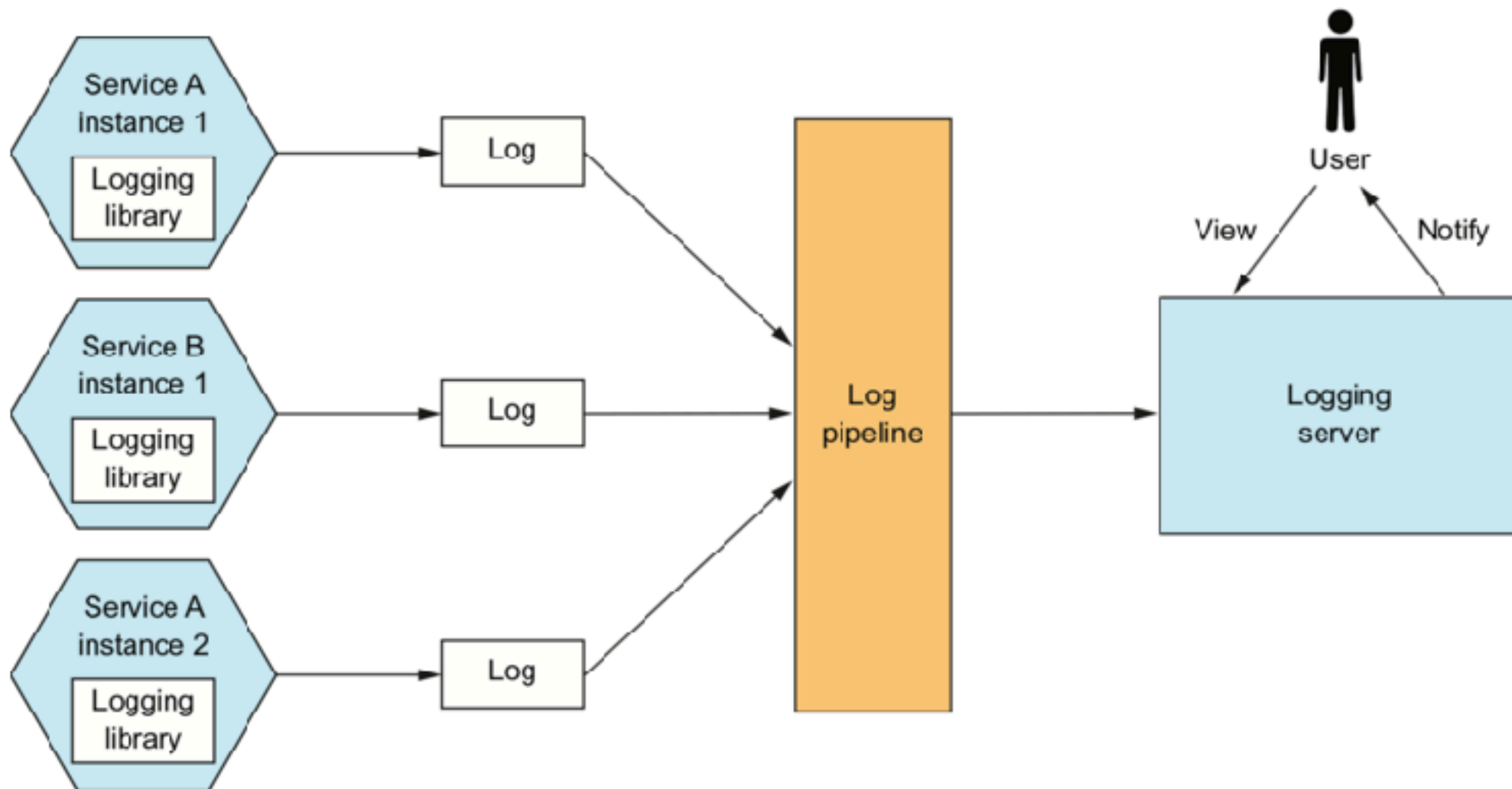


<https://www.jaegertracing.io>



Centralized logging

Log service activity and write logs into a centralized logging server. (searching, alerting)



Effective Logging

- Define event to log
- Use structured logging
- Exclude sensitive information
- Log at the correct level
- Be specific in your message
- Don't log large message
- Make sure you keep trace Id in the log

https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Logging_Cheat_Sheet.md



Consistent Structure across all logs

Property	Description	Example
Timestamp	Date and time of the log	2023-07-01
Log level	DEBUG, INFO, ERROR	
Trace Id or Correlation Id	Unique identifier that refer to other logs from all services	
Event/Action Name	Identify to event or action of log	Authentication fail
Service ID/ Name	Identify to service	
Request path	Path for the request	/api/products

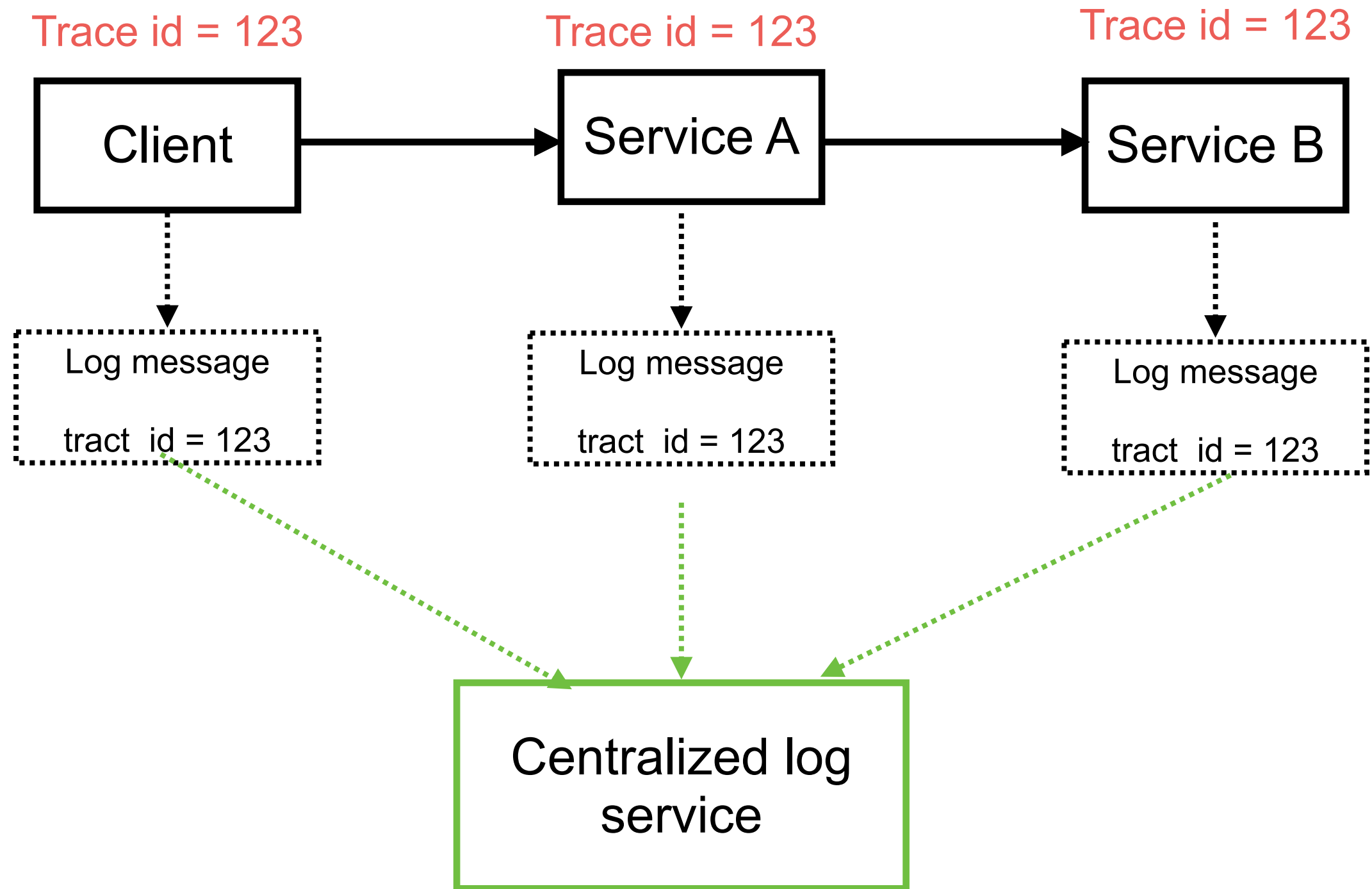


Don't keep !!

```
com.framework.FrameworkException: Error in web request
  at com.framework.ApplicationStarter.lambda$start$0(ApplicationStarter.java:15)
  at spark.RouteImpl$1.handle(RouteImpl.java:72)
  at spark.http.matching.Routes.execute(Routes.java:61)
  at spark.http.matching.MatcherFilter.doFilter(MatcherFilter.java:134)
  at spark.embeddedserver.jetty.JettyHandler.doHandle(JettyHandler.java:50)
  at org.eclipse.jetty.server.session.SessionHandler.doScope(SessionHandler.java:1568)
  at org.eclipse.jetty.server.handler.ScopedHandler.handle(ScopedHandler.java:144)
  at org.eclipse.jetty.server.handler.HandlerWrapper.handle(HandlerWrapper.java:132)
  at org.eclipse.jetty.server.Server.handle(Server.java:503)
  at org.eclipse.jetty.server.HttpChannel.handle(HttpChannel.java:364)
  at org.eclipse.jetty.server.HttpConnection.onFillable(HttpConnection.java:260)
  at org.eclipse.jetty.io.AbstractConnection$ReadCallback.succeeded(AbstractConnection.java:305)
  at org.eclipse.jetty.io.FillInterest.fillable(FillInterest.java:103)
  at org.eclipse.jetty.io.ChannelEndPoint$2.run(ChannelEndPoint.java:118)
  at org.eclipse.jetty.util.thread.QueuedThreadPool.runJob(QueuedThreadPool.java:765)
  at org.eclipse.jetty.util.thread.QueuedThreadPool$2.run(QueuedThreadPool.java:683)
  at java.base/java.lang.Thread.run(Thread.java:834)
Caused by: com.project.module.MyProjectFooBarException: The number of FooBars cannot be zero
  at com.project.module.MyProject.anotherMethod(MyProject.java:20)
  at com.project.module.MyProject.someMethod(MyProject.java:12)
  at com.framework.ApplicationStarter.lambda$start$0(ApplicationStarter.java:13)
  ... 16 more
Caused by: java.lang.ArithmeticException: The denominator must not be zero
  at org.apache.commons.lang3.math.Fraction.getFraction(Fraction.java:143)
  at com.project.module.MyProject.anotherMethod(MyProject.java:18)
  ... 18 more
```



Centralized logging



Q/A

